BABEŞ-BOLYAI UNIVERSITY CLUJ-NAPOCA
FACULTY OF MATHEMATICS AND COMPUTER SCIENCE
SPECIALIZATION COMPUTER SCIENCE IN ENGLISH

DIPLOMA THESIS

From Sound to Light: A Real-Time System for Generative Stage Lighting via Music Information Retrieval

Supervisor
Lect. PhD. Nechita Mihai

Author
Apăvăloaiei Alexandru

2026

UNIVERSITATEA BABEŞ-BOLYAI CLUJ-NAPOCA
FACULTATEA DE MATEMATICĂ ŞI INFORMATICĂ
SPECIALIZAREA INFORMATICĂ ENGLEZĂ

LUCRARE DE LICENŢĂ

De la sunet la lumină: un sistem în timp real pentru iluminat scenic generativ prin Music Information Retrieval

Conducător științific
Lect. dr. Nechita Mihai

Absolvent
Apăvăloaiei Alexandru

2026

ABSTRACT (EN)

Stage lighting is a fundamental component of live performance, shaping the emotional atmosphere and guiding the audience's perception of music in real time. Beyond mere visibility, light interacts with sound to amplify tension, convey mood, and create immersive experiences that resonate with spectators. As live events grow in scale and complexity, the demand for intelligent, music-responsive lighting systems has increased — motivating research at the boundary of audio analysis and automated visual design.

This thesis presents the design and development of a real-time generative stage-lighting system that transforms musical input into synchronized visual output. The work lies at the intersection of Music Information Retrieval, automatic stage lighting control, real-time audio processing, and interactive web-based 3D rendering. It builds on MIR techniques for extracting meaningful information from music, including rhythm and audio features, and applies them to a sound-to-light pipeline in which musical analysis directly drives lighting behavior.

The main contribution is the integration of research-oriented MIR and generative lighting models into an interactive application capable of operating as a practical real-time prototype. The system combines browser-based audio capture and 3D rendering with a Python inference server running BeatNet (beat detection and tracking) and Skip-BART (lighting cue generation), connected through WebSocket communication and shared frontend state. Its originality lies not in proposing a new MIR model, but in designing and implementing a coherent sound-to-light application that turns existing models into a responsive creative tool, with explicit attention to usability, latency, architecture, and expressive music-visual mapping.

ABSTRACT (RO)

Iluminatul scenic este o componentă fundamentală a spectacolului live, modelând atmosfera emoțională și ghidând percepția publicului asupra muzicii în timp real. Dincolo de simpla vizibilitate, lumina interacționează cu sunetul pentru a amplifica tensiunea, a transmite stări și a crea experiențe imersive care rezonează cu spectatorii. Pe măsură ce evenimentele live cresc în amploare și complexitate, cererea pentru sisteme de iluminat inteligente, reactive la muzică, a crescut — motivând cercetarea la granița dintre analiza audio și designul vizual automatizat.

Această lucrare prezintă proiectarea și dezvoltarea unui sistem generativ de iluminat scenic în timp real, care transformă intrarea muzicală în ieșire vizuală sincronizată. Lucrarea se află la intersecția dintre Music Information Retrieval, controlul automat al iluminatului scenic, procesarea audio în timp real și randarea 3D interactivă bazată pe web. Ea se bazează pe tehnici MIR pentru extragerea informațiilor relevante din muzică, inclusiv ritm și trăsături audio, și le aplică într-un flux sunet-lumină în care analiza muzicală conduce direct comportamentul luminilor.

Contribuția principală este integrarea modelelor MIR și a modelelor generative de iluminat, orientate spre cercetare, într-o aplicație interactivă capabilă să funcționeze ca prototip practic în timp real. Sistemul combină captură audio și randare 3D în browser cu un server Python de inferență care rulează BeatNet (detecția și urmărirea beat-ului) și Skip-BART (generarea indicațiilor de iluminat), conectate prin comunicație WebSocket și stare partajată în frontend. Originalitatea lucrării nu constă în propunerea unui model MIR nou, ci în proiectarea și implementarea unei aplicații sunet-lumină coerente care transformă modele existente într-un instrument creativ responsiv, cu atenție explicită la utilizabilitate, latență, arhitectură și mapare expresivă muzică-vizual.

Contents

Chapter 1

Introduction

1.1 Context and Motivation

Music performances are not experienced through sound alone. In live concerts, club environments, and electronic music events, lighting plays a central role in shaping atmosphere, directing attention, and amplifying emotional intensity. Changes in colour, brightness, movement, and timing can make a musical moment feel larger, more intimate, more aggressive, or more immersive. For electronic music in particular, where repetition, gradual build-ups, drops, and timbral changes are essential expressive devices, visual response becomes part of the perceived structure of the performance.

At the same time, expressive stage lighting is difficult to automate. Traditional lighting design depends on human expertise, preparation time, and detailed manual programming. Automatic Stage Lighting Control addresses this problem by generating lighting parameters from music, with the potential to support live performances, influence the audience's emotional experience, and provide a more accessible alternative to dedicated lighting engineers. However, many prior approaches rely on a two-stage strategy: first classifying music into predefined style or emotion categories, then assigning lighting patterns to those categories. Such pipelines can be useful, but they also reduce the richness of music to a limited set of labels before the visual output is generated.

Recent generative approaches suggest a more flexible direction. Instead of mapping a classified category to a fixed lighting rule, Skip-BART treats automatic stage lighting as a sequence generation problem and learns to produce lighting sequences directly from music. This framing is especially relevant for creative applications because lighting design is not only a technical control problem, but also an artistic process involving timing, continuity, contrast, and expressive variation. A system that can use such models in real time must therefore solve more than an inference

problem. It must capture or play audio, extract and stream meaningful signals, run analysis and generative models, transport outputs with low overhead, and update an interactive visual scene quickly enough for the result to feel synchronized with the music.

The motivation for this thesis stems from the convergence of two passions: programming and electronic dance music. Programming offers the means to build systems, structure complex processes, and turn abstract ideas into working applications. Electronic dance music provides a creative world where rhythm, energy, and atmosphere are inseparable from the visual experience — and where the connection between sound and light is felt immediately on the dance floor. This thesis was the natural opportunity to bring both together in a single project, exploring how music analysis and generative lighting models can be integrated into an application that is not only technically functional, but also visually engaging and suitable for interactive demonstration.

1.2 Objectives

The objective of the thesis is to design and implement a real-time sound-to-light prototype for generative stage lighting. The system analyzes audio while it is playing, extracts information relevant to musical timing and visual control, and uses model outputs to update a virtual stage environment. Beyond the technical pipeline, the goal is to create an environment in which a user can freely experiment — testing scene ideas, exploring lighting concepts, and interacting with the visual output as a creative tool rather than a static visualization. The full source code of the prototype is publicly available at `https://github.com/alexapvl/thesis_project`.

The application follows a hybrid architecture: the browser handles audio input, playback, user interaction, and 3D scene rendering, while a local server performs beat detection and generative lighting inference. Audio data is streamed from the browser to the server, and musical and lighting parameters are sent back to drive real-time visual updates.

A central design concern is real-time responsiveness, and it means different things for the two kinds of visual output the system produces. The thesis distinguishes a *beat-synchronization path*, in which detected beats drive immediate visual reactions, from a *generative lighting path*, in which Skip-BART produces the evolving colour palette. The system does not need to satisfy the extremely strict action-to-sound constraints of a musical instrument, but the beat-synchronization path must still keep visual reactions close enough to musical events to be perceived as synchronized; for this path the architecture targets a sub-100 ms end-to-end threshold and estimates it at approximately 90 ms. The generative lighting path operates on a coarser timescale

by design: because Skip-BART generates lighting for a whole audio window at once rather than streaming frame by frame, the colour palette refreshes every few seconds rather than within milliseconds. It is therefore governed by an update *cadence* rather than the sub-100 ms threshold—a distinction developed in Chapter 4 and quantified in Chapter 6. For this reason, the thesis treats system design as the management of latency, modularity, and usability under the constraints of the chosen platform.

1.3 Contributions

The original contribution of this thesis is the design, implementation, and evaluation of a complete real-time sound-to-light prototype that integrates beat detection and a generative lighting model into a single interactive application. The work does not propose a new machine-learning model; instead, its novelty lies in the system-level engineering required to make an offline generative model usable in a live, browser-based setting. The main contributions are the following:

- A **hybrid browser–server architecture** that separates real-time audio capture, user interaction, and 3D rendering in the browser from beat detection and generative lighting inference on a local server, connected through a low-overhead streaming protocol.
- The **integration of Skip-BART, an offline generative lighting model, into a live streaming pipeline**, including the windowing and cadence strategy required to drive a continuously evolving colour palette from a model that generates lighting one audio window at a time.
- A **dual-path real-time design** that explicitly separates a low-latency path for beat synchronization from a coarser-grained generative lighting path, each governed by its own responsiveness target.
- A **typed communication protocol** at the browser–server boundary that structures the exchange of audio data, musical parameters, and lighting outputs.
- An **empirical evaluation** of the prototype, benchmarking end-to-end latency, server-side processing, and inference cost across two hardware configurations.

The original results of the thesis are therefore primarily systemic and quantitative: a working prototype that demonstrates the feasibility of real-time generative stage lighting on the web, together with latency and performance measurements that characterise its behaviour. These results are presented in detail in Chapter 6.

1.4 Use of Generative AI Tools

In the interest of transparency, the author declares that generative AI tools were used in a limited and supporting capacity during the preparation of this thesis. Specifically, Claude (Anthropic) was used to improve the readability of the text, to rephrase passages that had already been written by the author, and to assist with selected aspects of software development, such as debugging, refactoring, and drafting small code fragments that were subsequently reviewed, adapted, and integrated by the author. These tools were not used to generate the original ideas, the system architecture, the research contributions, the experimental results, or the conclusions of this work, all of which are the author's own. Every AI-assisted suggestion, whether textual or programmatic, was reviewed, edited, and validated by the author, who takes full responsibility for the entire content of the thesis, for the source code of the prototype, and for the results obtained.

1.5 Thesis Structure

The thesis is organized as follows. Chapter 2 introduces the theoretical foundations of the work, covering digital audio and Music Information Retrieval, rhythmic structure, beat tracking, music-visual mapping, automated stage lighting control, real-time processing, and the deep-learning architectures relevant to the system. Chapter 3 evaluates the web as a platform for real-time audio-visual applications, examining rendering with WebGL and Three.js, audio processing through the Web Audio API, browser-based machine-learning deployment, communication mechanisms, frontend framework selection, and the design implications of platform constraints. Chapter 4 translates those constraints into a concrete system design by defining the methodological approach, requirements, hybrid architecture, module responsibilities, frontend data flow, and the end-to-end runtime pipeline. Chapter 5 describes how that design is realised in code, focusing on the engineering problems that most shaped the prototype: the typed protocol at the browser–server boundary, the integration of an offline generative model into a live streaming pipeline, the real-time beat path, and the separation between runtime and editable scene state. Chapter 6 evaluates the prototype empirically, benchmarking end-to-end latency, server-side processing, and inference cost across two hardware configurations. Chapter 7 concludes the thesis by consolidating the outcomes of the prototype and identifying the main lessons, limitations, and possible directions for future development.

Chapter 2

Theoretical Chapter

This chapter introduces the foundational concepts and theoretical frameworks that underpin the system presented in this thesis. It is organized to move from the general to the particular: beginning with how audio is represented and analyzed computationally, progressing through musical structure and beat tracking, the mapping of music to stage lighting, and the real-time constraints that shape the system's design, and concluding with the deep learning architectures and the sequence-to-sequence BART model on which the lighting generation depends. Two models in particular are central to the pipeline and are discussed throughout the chapter: BeatNet (BeatNet+ in the original publication), used for live beat and downbeat tracking, and Skip-BART, used for generating stage lighting sequences directly from music.

2.1 Music Information Retrieval Foundations

Music Information Retrieval (MIR) is a multidisciplinary research field concerned with extracting, analyzing, and organizing meaningful information from music. It draws on signal processing, machine learning, musicology, and psychoacoustics to address tasks such as beat tracking, chord recognition, genre classification, music recommendation, and emotion recognition[27]. The work presented in this thesis relies heavily on MIR techniques for analyzing audio signals in real time and extracting the musical features that drive the lighting generation pipeline.

At its core, MIR operates on digital representations of audio. Understanding how continuous sound is transformed into a computationally tractable signal, and how meaningful features are subsequently extracted from that signal, is essential to understanding everything that follows.

2.1.1 Digital Audio Signals

Sound in the physical world is a continuous variation in air pressure over time. To process sound computationally, this continuous signal must be converted into a discrete digital representation through a process known as analog-to-digital conversion. This involves two fundamental operations: sampling and quantization.

Sampling refers to measuring the amplitude of the continuous signal at regular time intervals. The rate at which these measurements are taken is called the sampling rate, measured in Hertz (Hz). A sampling rate of 44,100 Hz (44.1 kHz), the standard for CD-quality audio, means the signal's amplitude is recorded 44,100 times per second. The Nyquist-Shannon sampling theorem states that a sampling rate must be at least twice the highest frequency present in the signal to avoid aliasing, a form of distortion where high-frequency content is incorrectly represented as lower frequencies. Since the upper limit of human hearing is approximately 20 kHz, a 44.1 kHz sampling rate is sufficient to capture the full audible spectrum.

Quantization assigns each sampled amplitude value to the nearest level in a finite set of discrete values. The number of available levels is determined by the bit depth: 16-bit audio provides 65,536 discrete amplitude levels, while 24-bit audio provides over 16 million. Higher bit depth results in a more faithful representation of the original signal, with lower quantization noise.

The result of sampling and quantization is a sequence of numerical values, one per sample, that represents the audio waveform in the time domain. While this raw representation contains all the information in the signal, it does not directly reveal musically meaningful properties such as pitch, timbre, or rhythmic content. Extracting these requires transforming the signal into alternative representations.

2.1.2 The Short-Time Fourier Transform and Spectrograms

The most fundamental tool for analyzing the frequency content of an audio signal is the Fourier Transform, which decomposes a signal into its constituent sinusoidal components, revealing the frequencies present and their respective magnitudes. In discrete-time audio processing, the transform applied to a finite sequence of samples is the Discrete Fourier Transform (DFT):

$$X_k = \sum_{n=0}^{N-1} x_n \, e^{-i2\pi kn/N}, \qquad k = 0, \ldots, N-1$$

where x_n denotes the n-th sample in the window, N is the window length, and X_k is the complex coefficient corresponding to frequency bin k. In practice, this computation is performed efficiently using the Fast Fourier Transform (FFT), which makes it suitable for repeated application across many short windows. However,

the standard Fourier Transform operates on the entire signal at once, producing a single frequency spectrum that loses all temporal information. For music, where frequency content changes continuously over time, this is inadequate.

The Short-Time Fourier Transform (STFT) addresses this limitation by applying the Fourier Transform to short, overlapping segments (or 'windows') of the signal rather than to the signal as a whole. Each window, typically 20 to 50 milliseconds in duration, is assumed to be approximately stationary, meaning its frequency content does not change significantly within that short interval. Formally, the STFT can be written as

$$X(m,k) = \sum_{n=0}^{N-1} x[n+mH]\, w[n]\, e^{-i2\pi kn/N}$$

where $w[n]$ is a window function of length N, H is the hop size between successive windows, and m indexes the frame in time. The STFT slides this window across the signal with a defined hop size (the distance between successive windows), computing a frequency spectrum for each position. The result is a two-dimensional representation with time on one axis and frequency on the other, where the value at each point indicates the magnitude of a given frequency at a given moment. This representation is called a spectrogram.

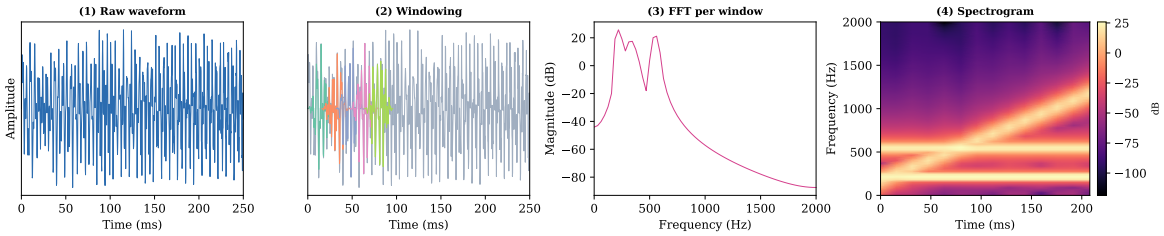

Figure 2.1: Audio-to-spectrogram pipeline: raw waveform (time domain), overlapping windowing, FFT on each window, and final time-frequency spectrogram representation.

The choice of window size involves a fundamental trade-off between temporal and frequency resolution. Shorter windows provide better temporal resolution (the ability to pinpoint when a frequency appears) but worse frequency resolution (the ability to distinguish between closely spaced frequencies). Longer windows provide the opposite. This trade-off is inherent to the uncertainty principle in signal processing and cannot be avoided, only managed according to the requirements of the task[27].

The spectrogram is a powerful representation because it makes the structure of audio visually apparent: sustained tones appear as horizontal bands, percussive events as vertical lines, and harmonic sounds as stacks of parallel bands at integer multiples of a fundamental frequency. However, the raw STFT spectrogram has a linear frequency axis, which does not correspond well to how humans perceive pitch.

In this thesis, the STFT — implemented as a sequence of FFT-computed DFTs over short windows — is the underlying analysis tool for all downstream representations, including the mel spectrograms used as input to the beat-tracking and lighting-generation models.

2.1.3 The Mel Scale and Mel Spectrograms

Human pitch perception is approximately logarithmic: the perceived difference between 100 Hz and 200 Hz is much greater than between 5,000 Hz and 5,100 Hz, even though the absolute difference in Hertz is the same. The mel scale is a perceptual scale of pitch that reflects this nonlinearity. It was derived from psychoacoustic experiments in which listeners were asked to judge equal intervals of pitch, and it maps frequencies so that equal distances on the mel scale correspond to equal perceived pitch intervals.

A mel spectrogram is obtained by projecting the frequency axis of a standard STFT spectrogram onto a set of mel-spaced triangular filter banks. The mel filter bank for projecting frequencies is inspired by the human auditory system and physiological findings on speech perception.[27] These filters are narrowly spaced at low frequencies (where human frequency discrimination is fine) and more widely spaced at high frequencies (where discrimination is coarser). The magnitudes within each filter band are summed, resulting in a reduced number of frequency bins that better reflect perceptual relevance. Applying a logarithmic transformation to these magnitudes yields the log-mel spectrogram, which further aligns the representation with human loudness perception.

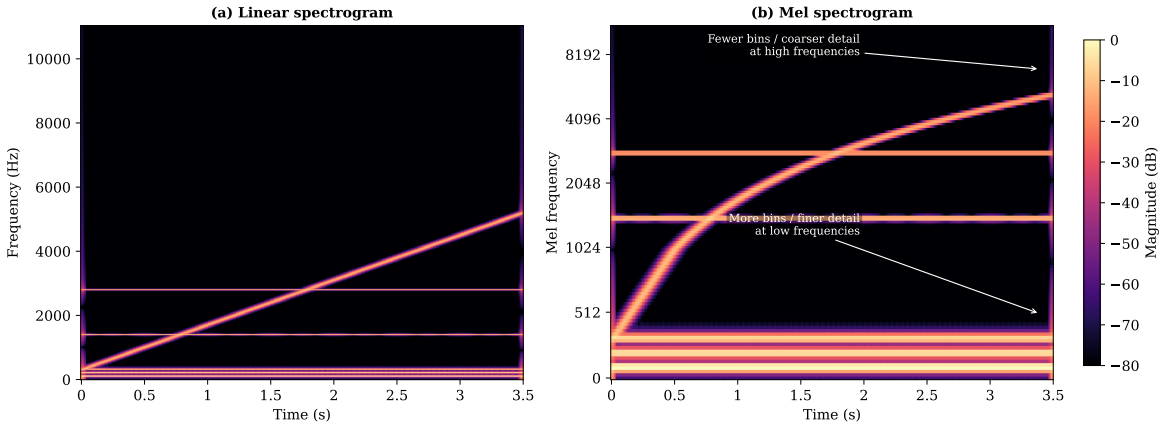

Figure 2.2: Comparison of two time-frequency representations of the same audio excerpt: linear-frequency STFT spectrogram (left) and mel-scaled spectrogram (right). The mel representation allocates finer resolution at low frequencies and coarser resolution at high frequencies, better reflecting human pitch perception.

The log-mel spectrogram has become the dominant input representation for deep learning models applied to audio.[27] It provides a compact, perceptually motivated

summary of the audio signal that retains the information most relevant to human hearing while discarding redundancy in the high-frequency range. Compared to raw waveforms, log-mel spectrograms require significantly less data and training time to achieve comparable classification performance, making them the practical default for most MIR tasks.[27]

2.1.4 Audio Features

While spectrograms provide a rich, general-purpose representation of audio, many MIR tasks benefit from extracting specific numerical descriptors, called audio features, that capture particular aspects of the signal. These features can be broadly categorized into spectral, temporal, and tonal features.

Spectral features describe the shape and characteristics of the frequency spectrum at a given moment. The spectral centroid represents the center of mass of the spectrum and correlates with the perceptual quality of brightness: sounds with energy concentrated at higher frequencies have a higher spectral centroid. The spectral bandwidth measures the dispersion of energy around the centroid, indicating timbral richness. Spectral flux quantifies how much the spectrum changes between consecutive frames, making it particularly useful for detecting onsets and transient events in music. The spectral rolloff is the frequency below which a specified percentage (typically 85% or 95%) of the spectral energy is concentrated, serving as an indicator of the spectral shape and harmonicity of the sound. The spectral contrast captures the dynamic range between peaks and troughs across frequency subbands, providing information about timbral texture.

Mel-Frequency Cepstral Coefficients (MFCCs) are a compact representation of the spectral envelope. They are computed by taking the discrete cosine transform (DCT) of the log-mel spectrum, which decorrelates the filter bank energies and compresses the information into a small number of coefficients (typically 13 to 20). For decades, MFCCs were the dominant feature representation in audio analysis, particularly in speech recognition.[27] In the context of deep learning, their use has declined in favor of log-mel spectrograms, since the DCT step that produces MFCCs discards spatial relationships between frequency bands that convolutional neural networks can exploit directly.[27] Nevertheless, MFCCs remain a standard feature in many MIR pipelines and provide a useful, low-dimensional summary of timbral characteristics.

Temporal features describe properties of the signal in the time domain. The zero-crossing rate (ZCR) measures how frequently the signal changes sign, which correlates with the noisiness or percussive character of the sound. Root mean square (RMS) energy provides a frame-level measure of loudness. Onset strength functions

measure the likelihood of a musical onset (the beginning of a note or percussive hit) at each time step, typically derived from the rate of increase in spectral energy.

Tonal and chroma features capture pitch-class information independently of octave. A chroma feature vector represents the energy distribution across the twelve pitch classes of the Western chromatic scale (C, C#, D, ..., B), collapsing octave information so that all instances of a given note class contribute to the same bin. Chroma features can be derived from the STFT (Chroma STFT), the Constant-Q Transform (Chroma CQT, which offers better frequency resolution at low frequencies), or computed with energy normalization and temporal smoothing (Chroma CENS) for more robust harmonic pattern recognition. These features are particularly relevant for tasks involving harmonic analysis, chord recognition, and key estimation.

Summary of Core Audio Feature Categories in MIR

Feature Category	Representative Features	Musical Quality Captured
Spectral	Centroid, bandwidth, flux, rolloff, contrast	Brightness, timbral shape, onset/transient change
Cepstral	MFCCs (typically 13-20 coefficients)	Spectral envelope / timbre summary
Temporal	Zero-crossing rate, RMS energy, onset strength	Noisiness/percussiveness, loudness, event salience
Tonal	Chroma (STFT/CQT/CENS)	Harmonic content, chord/key tendencies

Figure 2.3: Summary of major audio feature categories used in MIR, with representative descriptors and the corresponding musical qualities they capture.

The choice of which features to extract depends on the task at hand. Beat tracking systems typically operate on onset strength functions or spectral flux, while systems concerned with mood or energy may rely more heavily on spectral centroid, loudness curves, and MFCCs. For the system described in this thesis, which must translate audio into lighting parameters that reflect both rhythmic and timbral qualities of the music, a combination of spectral, temporal, and rhythmic features is relevant. The specific feature extraction pipeline is discussed in Chapter 5.

2.2 Musical Structure and Beat Tracking

The previous section established how raw audio can be transformed into feature representations that capture different musical qualities. This section turns to the musical structures those features are ultimately meant to describe: rhythm, meter, and form. It begins with the fundamentals of rhythmic organisation, moves to the specific structural conventions of electronic dance music, and then formalises beat

tracking as a computational task.

2.2.1 Rhythmic Fundamentals

Music unfolds as a sequence of events in time, and the most basic organising principle of that temporal flow is the beat: a perceived regular pulse that listeners naturally tap along to. The rate at which beats occur is the tempo, conventionally expressed in beats per minute (BPM). A typical pop or rock track might sit around 100 to 130 BPM, while EDM tempos cluster between roughly 120 and 130 BPM for genres like house and progressive house, and may reach 140 BPM or higher in trance and drum-and-bass.

Not all beats carry equal perceptual weight. Western music groups beats into recurring patterns of strong and weak pulses called meter. The first beat of each group, the downbeat, is perceived as metrically strongest. The grouping itself defines the time signature: a 4/4 meter groups four beats per measure (or bar), with the downbeat on beat one; a 3/4 meter groups three. These patterns create a metrical hierarchy, a layered grid ranging from sub-beat divisions (such as eighth or sixteenth notes) up through beats, downbeats, and multi-bar phrases. Listeners unconsciously infer this hierarchy from the acoustic signal, and it guides expectations about when the next salient event will occur.

This hierarchical view matters for computational systems because different applications need different levels of the grid. A simple visualiser that flashes on every beat needs only the beat level. A lighting controller that distinguishes verse from chorus needs the downbeat level (to know where measures begin) and, ideally, a sense of higher-level phrasing (to know where sections change). Beat tracking and downbeat tracking are therefore treated as distinct but related tasks in MIR.

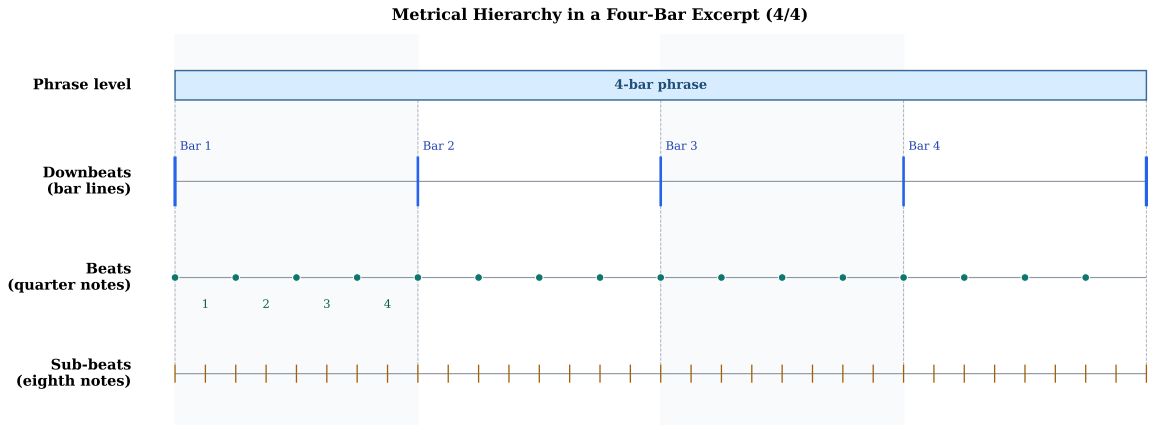

Figure 2.4: Metrical hierarchy in a four-bar 4/4 excerpt. The diagram shows nested rhythmic levels: phrase boundary (top), downbeats at bar lines, quarter-note beats, and eighth-note sub-beat divisions.

2.2.2 EDM Form and Structure

Electronic dance music occupies a distinctive position in the landscape of popular music. Its rhythmic foundation is overwhelmingly grid-based: sequences are quantised to exact metric positions, and the dominant rhythmic pattern is the four-on-the-floor kick drum, a bass drum striking on every beat of a 4/4 bar. Brøvig-Hanssen et al. observe that EDM "literally sound[s] like a machine" by presenting musical patterns that "act like machines, and by cultivating aesthetic qualities that we associate with machines (such as precision [and] speed)."[29] This grid-based precision might seem to leave little room for expressive nuance, yet producers manipulate sonic features such as attack shape, sidechain compression, and swing quantisation to introduce subtle rhythmic friction that makes the groove feel alive rather than mechanical.

At the macro level, EDM tracks follow a broadly predictable formal scheme. Butler (2006) and Snoman (2009) identify a conventional template in which a track contains two cycles of build-up and drop, with the second cycle typically more intense than the first.[29] Solberg (2014) refines this into a framework centred on three key sections: the breakdown, the build-up, and the drop. These names reflect their musical function. The breakdown strips the track's texture down, removing layers and, crucially, removing the bass and bass drum, the rhythmic foundation on which dancers rely and coordinate themselves. The build-up that follows gives strong indications of a massive musical, emotional, and bodily peak ahead. The drop then reintroduces the bass and bass drum, resolving the accumulated tension in what is often the most intense moment of the track.[29] A complete track typically frames these cycles within an intro and outro that facilitate DJ mixing.

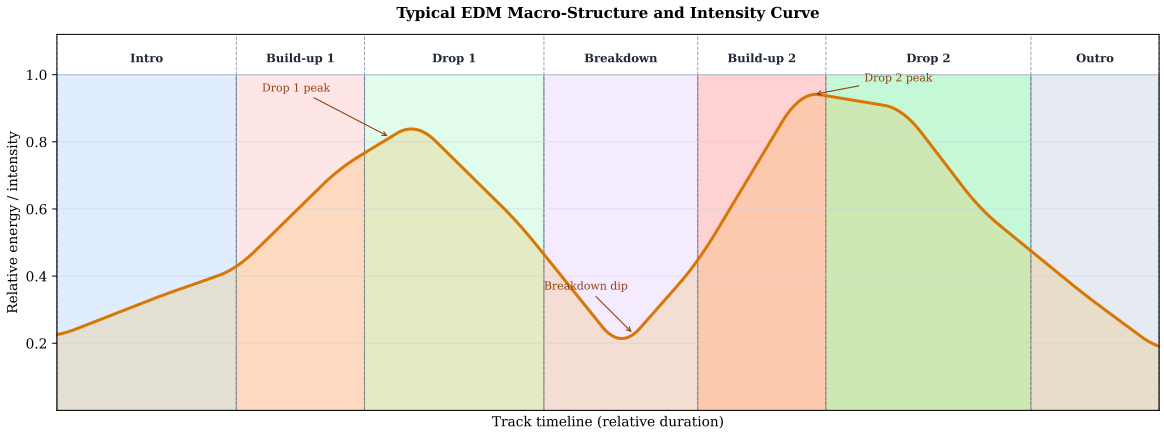

Figure 2.5: Typical EDM macro-structure shown as a timeline of sections (Intro, Build-up 1, Drop 1, Breakdown, Build-up 2, Drop 2, Outro), with an overlaid intensity curve that dips during the breakdown and peaks at the drops. Adapted from structural descriptions by Butler (2006) and Solberg (2014).

What makes this formal structure audible to listeners (and potentially detectable

by computational systems) is a set of production techniques that function as structural markers. Solberg identifies five dominant techniques used in build-ups and drops: (i) extensive use of uplifters, rising noise sweeps or pitched sounds that create a sensation of upward motion; (ii) the "drum roll effect," a rhythmic acceleration of percussive hits as the drop approaches; (iii) large frequency changes, often achieved through filter sweeps; (iv) the removal and reintroduction of the bass and bass drum; and (v) a contrasting breakdown that heightens the impact of the subsequent drop.[29]

Smith (2021) provides a complementary analytical framework by focusing specifically on continuous processes: musical gestures characterised by continuous changes to parameters, as opposed to discrete, step-by-step ones. Examples include pitch slides (glissandos), crescendos, fade-ins, accelerandos, filter sweeps, and timbre changes accomplished through manipulation of sound envelopes and frequency content. These processes are created through continuous controllers such as sliders and knobs, or programmed into tracks with automation curves. Functionally, Smith argues, continuous processes serve three roles. In core sections they provide ornamentation, embellishing rhythms and melodies. In build-up sections they provide orientation, often accompanied by intensification, guiding listeners toward structural boundaries so they can anticipate the drop and coordinate their bodies to the beat. In breakdown sections they can produce disorientation, creating sounds that are hard to interpret within the prevailing meter and tempo, signalling to dancers that they can take a break.[28]

Together, these structural and production conventions mean that EDM tracks, despite their repetitive surface, contain clear hierarchical organisation at multiple time scales: sub-beat rhythmic patterns, four-on-the-floor metric regularity, multi-bar phrases, and section-level formal structure marked by systematic changes in energy, spectral content, and textural density. This layered organisation is what an automated music analysis system must capture if it is to drive meaningful visual responses.

2.2.3 Beat Tracking

Beat tracking, the task of estimating the temporal locations of musical beats in an audio signal, is one of the long-standing problems in MIR.[8] It is often combined with downbeat tracking, which targets the higher metrical level of identifying the beginning of each measure. Both tasks are fundamental to any system that needs to synchronise visual output with music.

Historically, beat tracking grew out of onset detection, the task of identifying starting points of musically relevant events such as notes. Early onset detectors

used hand-designed spectral features like spectral flux to detect sudden changes in the frequency content of a signal. However, modern beat tracking systems tackle the task more directly without relying on a separate onset detection stage.[27]

Most recent systems follow a common pipeline consisting of three stages. First, the audio is transformed into a time-frequency representation, typically a mel spectrogram. Second, a neural network (often a convolutional recurrent neural network, or CRNN) processes the spectrogram to produce frame-wise beat and downbeat activation values: a probability for each time frame indicating how likely it is to contain a beat or downbeat. Third, a post-processing stage converts these noisy, continuous activations into discrete beat positions.[8]

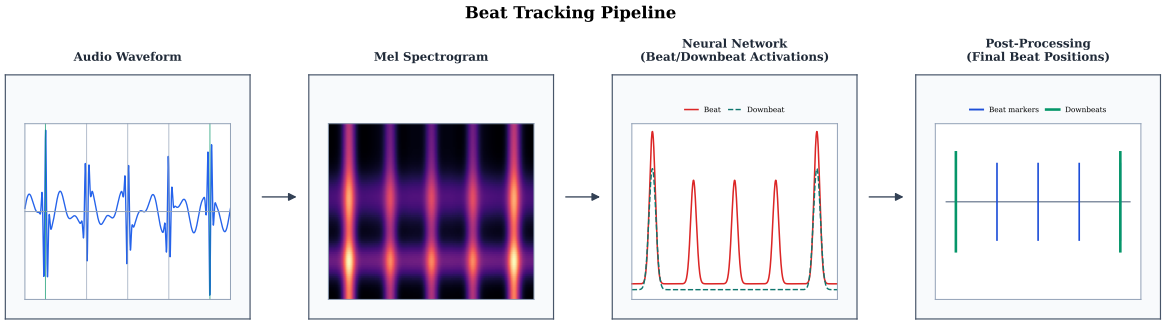

Figure 2.6: Beat tracking pipeline: audio waveform to mel spectrogram, neural network beat/downbeat activations, and post-processing that converts activations into discrete beat positions on a timeline.

The most widely used post-processing method is the Dynamic Bayesian Network (DBN) in the form proposed by Böck et al. (2011). The DBN imposes a probabilistic model over tempo and meter, forcing the predicted beats to conform to musically plausible constraints: a tempo range (typically 55 to 215 BPM), a set of allowed time signatures (typically 3/4 and 4/4), and a degree of tempo continuity.[8] This works well for the majority of popular music, where tempo and meter are stable, but it can fail on pieces that deviate from these assumptions, such as music with tempo changes, unusual time signatures, or rubato performance. An alternative post-processing paradigm is particle filtering, used by systems such as BeatNet. In particle filtering, a set of particles represents and evolves the posterior distribution of rhythmic states (beat, downbeat, non-beat) over time, allowing the system to track tempo changes more flexibly than the DBN. BeatNet combines a CRNN front-end with a two-level cascade Monte Carlo particle filter to perform real-time joint beat, downbeat, and meter tracking.[10]

A critical distinction for real-time applications is between offline and online beat tracking. Offline systems have access to the entire audio recording and can use bidirectional models and global optimisation, yielding the highest accuracy. Online (or causal) systems must produce beat estimates using only past and present audio

frames, which is substantially harder. Recent advances in online tracking include BEAST (Chang and Su, 2024), which applies a streaming Transformer encoder with contextual block processing to produce beat and downbeat activations from partially available audio frames, achieving an F1 score of 80.04% for beats under a 46-millisecond latency constraint.[5] The PLP-based system of Meier et al. (2024) takes a different approach, building on the predominant local pulse concept to achieve zero-latency beat tracking with a lookahead mechanism that can predict beats several hundred milliseconds in advance.[18] At the offline end of the spectrum, Foscarin et al. (2024) demonstrate that a carefully designed neural architecture combining convolutions and transformers can achieve state-of-the-art F1 scores (92.6% for beats, 85.4% for downbeats) without any DBN post-processing at all, using instead a minimal peak-picking strategy and a novel "sum head" that ensures downbeat predictions always coincide with beat predictions.[8]

Beat tracking performance is evaluated using several standard metrics. The F1 score, computed with a tolerance window (typically ±70 milliseconds), measures the proportion of correctly detected beats. Two additional continuity-based metrics, CMLt (Correct Metrical Level, continuity-required) and AMLt (Allowed Metrical Levels, continuity-required), impose a stricter criterion: a beat is counted as correct only if both it and the preceding beat are correctly placed, ensuring that the tracker maintains a consistent pulse rather than producing isolated correct hits. AMLt further allows predictions at alternative metrical levels (such as double or half tempo, or offbeats), accommodating the inherent ambiguity of metrical interpretation.[8] For a real-time audio-visual system like the one developed in this thesis, the F1 score and latency are the most practically relevant metrics: the system must detect beats accurately and quickly enough for the visual response to feel synchronised with the music.

2.3 Stage Lighting and Music-Visual Mapping

The previous sections have established how audio is represented and analysed (Section 2.1) and how musical structure is defined and detected (Section 2.2). This section turns to the output side of the pipeline: stage lighting. It examines the colour model used to represent lighting states, the perceptual and psychological evidence for systematic associations between music and colour, and the evolution of automated stage lighting control from rule-based approaches to the generative paradigm adopted in this thesis.

2.3.1 The HSV Colour Model

Stage lighting is most naturally described in terms of colour and brightness rather than the red-green-blue (RGB) channel decomposition used internally by display hardware. The HSV (Hue, Saturation, Value) colour space provides exactly this separation. Hue encodes the colour itself as an angle on a colour wheel, Saturation encodes the purity or vividness of the colour, and Value encodes its brightness. This decomposition makes the control and generation of lighting more intuitive, independent, and stable, because a lighting designer (or a generative model) can adjust brightness without affecting colour, or shift colour without changing intensity.[34]

In the implementation used by Zhao et al. (2025) for the Skip-BART model, the HSV representation is applied following the OpenCV convention. The Hue channel ranges from 0 to 179, representing 180 discrete hue categories on a cyclic scale (where 0 and 179 are adjacent, both corresponding to red). The Value channel ranges from 0 to 255, representing 256 brightness levels from completely dark to maximum intensity. The Saturation channel is fixed at 255 (100%) to simulate vivid stage lighting and counteract environmental influences such as air scattering, fog, or surface reflections that reduce colour purity. This simplification reduces the lighting control problem to two channels, hue and value, while preserving the perceptually most important dimensions for stage lighting design.[34]

The cyclic nature of the hue channel presents a specific modelling challenge: the values 0 and 179 are numerically distant but perceptually adjacent (both are red). As discussed in Section 2.6, this is why Skip-BART uses a discrete tokenisation and embedding approach rather than treating hue as a continuous value, since an embedding layer can learn the circular topology that a standard regression head cannot.[34]

2.3.2 Music-Colour Associations and Emotional Mediation

The decision to generate coloured lighting from music rests on the assumption that there are systematic, non-arbitrary associations between musical qualities and colours. Experimental evidence supports this assumption. Palmer et al. (2013) conducted a series of experiments in which non-synesthetic participants from the United States and Mexico were asked to choose colours that were most and least consistent with 18 selections of classical orchestral music by Bach, Mozart, and Brahms.[22] The musical selections varied systematically in tempo (slow, medium, fast), mode (major, minor), and composer.

The results revealed robust cross-modal associations between specific musical dimensions and specific colour dimensions. Faster music in the major mode was generally associated with more saturated, lighter, and yellower colours, whereas

slower music in the minor mode was associated with more desaturated (greyer), darker, and bluer colours. Critically, these associations were strongly consistent across both the US and Mexican participant groups, suggesting a high degree of cultural invariance.[22]

To explain these associations, Palmer et al. propose the emotional mediation hypothesis: listeners experience an emotional response to the music, and then select colours whose emotional connotations match that response.[22] Separate multidimensional scaling (MDS) analyses of the emotional associations of colours and of musical selections revealed that both could be well explained by the same two emotional dimensions: happy/sad (positive/negative valence) and strong/weak (high/low potency). The correlations between the emotional ratings for the musical selections and the emotional ratings for the colours chosen to go with them were remarkably high: +0.97 for happy/sad, +0.99 for strong/weak, +0.96 for lively/dreary, and +0.89 for angry/calm.[22] Two additional experiments, one mapping colours to emotional facial expressions and another mapping music to emotional faces, provided further converging evidence for the emotional mediation account.[22]

These music-colour associations are distinct from synesthesia. While a small minority of individuals report involuntary cross-modal experiences of colour while hearing musical sounds, scientific studies initially failed to establish general correspondences because synesthetic sound-to-colour mappings appeared idiosyncratic.[22] The associations reported by Palmer et al. are, by contrast, widely shared across non-synesthetic populations. Separately, non-synesthetic people also exhibit relatively low-level auditory-visual associations: higher pitches tend to be associated with lighter colours, and there is evidence for other correspondences such as timbre-saturation, loudness-brightness, and pitch-size associations. Spence (2011) argues that three types of cross-modal matching mechanisms have been established empirically: structural correspondences based on common neural coding, statistical correspondences based on natural covariation of cross-modal attributes, and semantically mediated correspondences based on common descriptive terminology.[22] Palmer et al.'s results provide clear evidence for a fourth type: cross-modal correspondences based on common emotional associations.[22]

For the system developed in this thesis, the emotional mediation framework provides a principled theoretical basis for why music-driven lighting should produce perceptually meaningful results. If listeners consistently associate certain musical qualities with certain colours via shared emotional connotations, then a model trained on real lighting data created by professional lighting engineers should, in principle, learn to reproduce these associations, even without an explicit emotion classification stage.

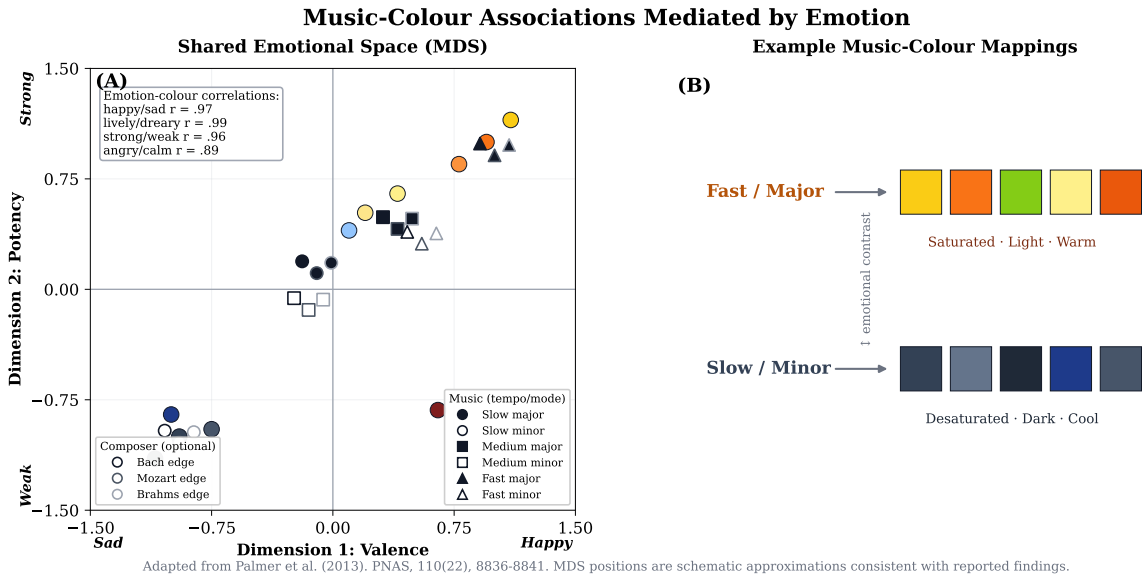

Figure 2.7: Music-colour associations mediated by emotion, adapted from Palmer et al. (2013): (A) a shared emotional space (MDS) showing alignment between colour and music selections along valence (sad-happy) and potency (weak-strong), with reported emotion-colour correlations; and (B) schematic mappings from fast/major music to saturated warm colours and slow/minor music to desaturated cool colours.

2.3.3 Automated Stage Lighting Control

Automatic Stage Lighting Control (ASLC) refers to the computational task of generating lighting parameters from music, and it shows potential in live performances by shaping the atmosphere and influencing the emotions of both musicians and audiences, while offering an affordable alternative to human lighting engineers. Although ASLC is not a widely explored topic in MIR, it has received attention in several prior works.[34]

Most prior methods follow a two-stage paradigm that first classifies music into predefined categories, such as emotion or style, and then defines a corresponding light pattern for each category.[34] Style-based approaches (Stanescu et al., 2018; Lei et al., 2021) and emotion-based approaches (Hsiao et al., 2017; Moon et al., 2015) represent the two mainstream classification strategies.[34] Emotion classification methods have employed various machine learning techniques, including Support Vector Machines, Support Vector Regression, and Multi-Layer Perceptrons.[34] For the light mapping stage, Hsiao et al. (2017) map music to emotional categories and infer specific lighting patterns based on embeddings in emotional space, while Nijdam (2009) provides a mapping that takes into account both hue and value.[34] Other approaches classify music using lyrics (Liao et al., 2023) or chords (Mabpa et al., 2021).[34]

These rule-based methods exhibit significant limitations. First, coarse-grained classification severely impacts model performance: categories are typically too broad

to capture the nuance of real lighting design.[34] Second, there is insufficient theoretical support for deterministically mapping these categories to specific light patterns, since lighting is an amalgam of technology and art rather than a deterministic function.[34] Third, the performance of all rule-based methods depends on the accuracy of the upstream classifier, and emotion classification accuracies in many studies fall below 80%, raising doubts about the practicality of these methods at their current stage.[34] Furthermore, many pieces of music lack lyrics or contain lyrics that are difficult to recognise (such as screaming in heavy music), rendering lyrics-based methods ineffective for large portions of the musical repertoire.[34]

Zhao et al. (2025) propose a fundamentally different approach by treating ASLC as a generative task.[34] Rather than classifying music and then applying predefined mappings, Skip-BART is trained end-to-end on data from professional lighting engineers, learning to generate lighting sequences directly from music in a single pass.[34] This generative framing avoids the information bottleneck inherent in classification-based pipelines and allows the model to learn the full complexity and artistry of professional lighting design. Through both quantitative analysis and human evaluation, Skip-BART demonstrates performance closely matching that of real human lighting engineers while significantly outperforming conventional rule-based methods across all evaluation metrics.[34]

The concepts introduced in this section establish the output representation and perceptual rationale for music-driven lighting. The next section examines real-time constraints, after which the chapter returns to the model architectures used for audio analysis and sequence generation.

2.4 Real-Time Audio Processing

The system described in this thesis must analyse audio and generate lighting responses while music is playing, not after it has finished. This requirement places the system within a class of audio applications that operate under real-time constraints, where the speed of processing is not merely a convenience but a functional necessity. This section defines what real-time operation means in the context of audio processing, examines the sources of latency that constrain system design, and considers the implications of these constraints for the present work.

2.4.1 Streaming, Block Processing, and Causality

Audio algorithms designed for real-time interaction cannot wait for an entire input signal before producing output. Instead, they must process sequential blocks of one or more audio samples at a time, an approach known as block processing or stream-

ing. In generative or analytical models built from multiple neural network layers, every layer must support this scheme: each layer is required to keep track of its internal state between subsequent forward passes to guarantee a continuous output signal between blocks.[4] For convolutional layers, this is typically achieved through a cached padding mechanism in which the end of the input tensor is retained and used to pad the next input.

A central distinction in streaming systems is between causal and non-causal processing. The outputs of causal systems depend only on current and past inputs, which is a strict requirement for truly real-time streaming models. Non-causal systems, by contrast, incorporate a finite number of future input timesteps, a lookahead, that improves reconstruction or prediction quality at the cost of additional delay. Non-causal models can be reconfigured for causal operation by introducing delays that allow the required future samples to be acquired before processing, but this conversion comes at a latency cost. Although non-causal models typically afford better reconstruction quality, in latency-sensitive applications it is desirable to avoid this additional cumulative delay by training in a causal manner from the outset.[4]

2.4.2 Real-Time Factor

The standard metric for assessing whether an algorithm can operate in real time is the Real-Time Factor (RTF), defined as the ratio between processing time and the duration of the input: RTF = t_p/t_i (t_p is the processing time and t_i is the input time).[4] An RTF below 1 means the algorithm processes audio faster than it arrives, which is the minimum condition for real-time operation. However, an RTF just below 1 may still be insufficient in practice: low-latency models must process very short audio blocks at a high rate, which severely limits the available computation time for a single inference pass.[4]

RTF alone does not fully characterise suitability for live use. The Lyria Team at Google DeepMind distinguish live music models from merely fast offline ones by requiring three attributes simultaneously: real-time throughput (RTF $\geq$ 1x), causal streaming in which audio is generated continuously as a function of both user control inputs and past audio output, and responsive controls where the delay between input and output is low enough to facilitate live interaction.[12] This tripartite definition is instructive because it highlights that speed (RTF) is a necessary but not sufficient condition for real-time interaction; the processing must also be causal and the end-to-end delay must be perceptually acceptable.

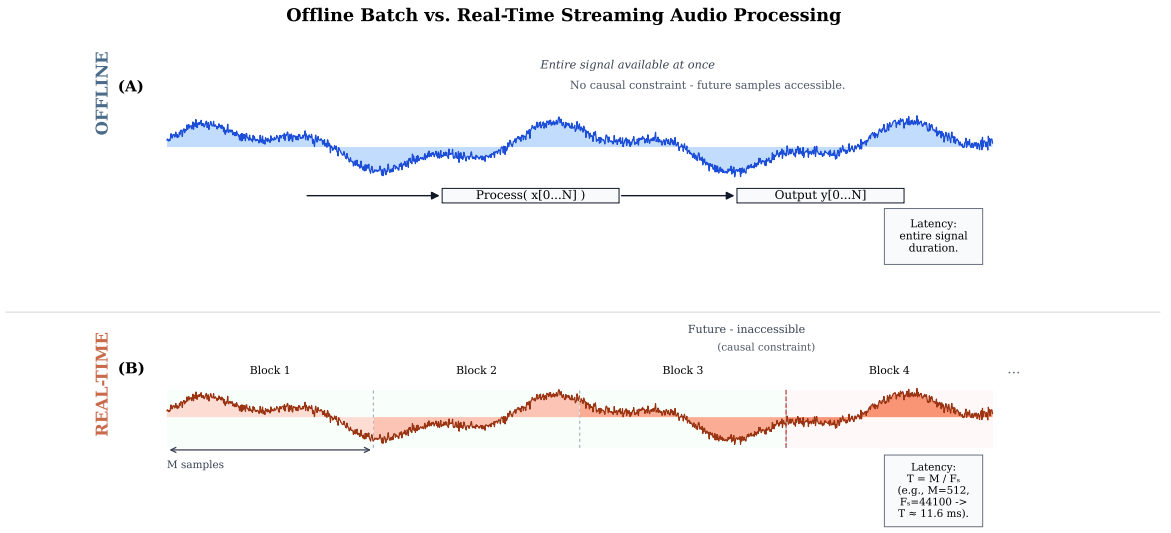

Figure 2.8: Comparison of offline batch processing and real-time streaming audio processing. Offline processing consumes the full signal at once with unrestricted access to future samples. In streaming mode, audio arrives in sequential blocks of size M at sample rate F_s, giving frame period $T = M/F_s$ (e.g., $M = 512$, $F_s = 44100 \Rightarrow T \approx 11.6$ ms), and outputs are constrained to depend only on past and present input.

2.4.3 Sources of Latency

Achieving an acceptable RTF does not guarantee low latency. Caspe et al. (2025) present a systematic taxonomy of five architectural sources of latency in interactive audio-to-audio neural systems, providing a useful framework for understanding how delay accumulates in real-time audio processing pipelines.[4]

Buffering delay arises from the double-buffering approach common to real-time audio systems, in which one audio block is computed while the next is being captured. The total output buffering latency is therefore two block lengths: one for sample acquisition and one for computing.[4] The block size therefore sets a hard floor on latency and simultaneously determines the upper limit for computation time per block.

Cumulative delay arises when non-causal convolutional layers are reconfigured for streaming by adding a delay line that allows the layers to look ahead in time. This lag accumulates through successive layers and can increase latency in non-causally trained models by hundreds of milliseconds compared to the same architecture trained causally.[4]

Representation delay refers to the number of samples required for a feature extractor to produce its expected output. Audio models commonly use feature representations such as Pseudo-Quadrature Mirror Filters (PQMF), Short-Time Fourier Transforms (STFT), or Mel filterbanks, each of which introduces its own processing delay.[4] Linear-phase Finite Impulse Response filterbanks, for instance, exhibit a

group delay of approximately half the filter length.

Data-dependent latency is a less widely recognised source. Convolutional layers are effectively FIR filters whose coefficients are learned through training. Models based on convolutional networks can therefore exhibit different degrees of latency depending on the learned coefficients, which are ultimately shaped by the training data. Caspe et al. empirically observed that signals with strong transients, such as percussive drum recordings, resulted in models that prioritised temporal accuracy, yielding lower latency and jitter compared to models trained on sustained harmonic content.[4]

Finally, positional uncertainty, or jitter, arises from lossy representations computed using block processing. Because the exact position in samples of an event is lost within a block, the model's response depends not only on the input and training method but also on the relative position of the event within a block, introducing a temporal uncertainty of up to one block size.

2.4.4 Latency Thresholds and Perceptual Constraints

The acceptable level of latency depends on the application. For instrumental musical interaction, where a performer's action must produce an audible response that feels immediate, the constraints are particularly strict. Wessel and Wright suggest an upper bound on latency of 10 ms and a jitter of ±1 ms for musical interaction. Empirical studies have supported this recommendation, demonstrating that 10 ms of latency was acceptable to performers in the absence of jitter, while 20 ms of latency or 10 ± 3 ms with jitter significantly degraded the performance experience. The sensitivity of musicians to latency also varies by instrument and monitoring setup, reinforcing the context-dependent nature of this threshold.[4]

Not all real-time audio applications require instrumental-grade latency, however. Spoken dialogue systems, for example, operate at considerably higher latencies: the Moshi framework uses causal models to encode and decode audio frames of 80 ms, yielding a final latency of 200 ms, which is tolerable for conversational interaction but far above the instrumental threshold.[4] This range illustrates the principle that latency requirements are fundamentally shaped by the perceptual expectations of the interaction: the tighter the feedback loop between action and response, the lower the tolerable delay.

2.4.5 Latency Budgets and Design Trade-offs

In practice, the total latency of a real-time system is the sum of contributions from each stage in its processing pipeline. Understanding how latency accumulates is therefore essential for system design. Caspe et al. (2025) illustrate this through a

detailed analysis of the RAVE neural audio synthesis architecture, revealing how design choices that prioritise reconstruction quality can make a system unsuitable for low-latency interaction. In RAVE's default configuration, the buffering delay alone accounts for 92.9 ms (two blocks of 2048 samples at 44.1 kHz), representation delay from the PQMF filterbank adds 12 ms, and the encoder's high compression ratio of 2048 introduces a jitter span of ±23.2 ms.[4] When non-causal streaming is enabled, the cumulative delay adds a further 566 ms.[4] The very features that make RAVE attractive for automatic music generation, namely its high compression ratio and non-causal mode of operation, make it highly unsuitable for low-latency interaction.

The RAVE case study demonstrates that latency reduction requires addressing each source individually and accepting the associated trade-offs. By progressively halving RAVE's compression ratio down to 128 and training causally, Caspe et al. achieved a buffering delay of 5.8 ms with a jitter of ±2.9 ms, approaching their target of 10 ± 1 ms for instrumental interaction.[4] The broader lesson is that designing for low latency requires putting this issue at the centre of the design problem, carefully inspecting audio representations and architecture to ensure they do not introduce unnecessary delay.[4]

2.4.6 Implications for the Present System

The latency constraints facing the system developed in this thesis differ from those of instrumental interaction but remain non-trivial. The system does not close an action-to-sound loop in the sense of a digital musical instrument; rather, it analyses incoming audio and produces a visual (lighting) response. Caspe et al. draw a useful distinction between action-to-sound latency, the delay between a performer's gesture and the resulting sound, and sound-to-sound latency, the delay between a captured audio input and the system's output.[4] The present system operates in a sound-to-light paradigm, where the perceptual tolerance for delay is more generous than for instrumental control but still bounded by the audience's expectation that lighting changes should appear synchronised with musical events.

The total latency budget of the system is the sum of its processing stages: audio capture and buffering, feature extraction (including beat tracking and onset detection), the Skip-BART model's inference for colour prediction, and the rendering of the visual output. Each of these stages contributes its own delay. Beat tracking algorithms, for instance, introduce latency determined primarily by the hop size of their feature extraction and, in the case of transformer-based models, by the look-ahead window. The BEAST streaming beat tracker reports an algorithmic latency of 46 ms at its lowest setting (with a single look-ahead frame), achieving an RTF of

0.41, while the BeatNet system operates at 20 ms latency with an RTF of 0.05.[5] The latency of these models is defined as the hop size of the STFT used for feature extraction. There is a direct trade-off between latency and accuracy: BEAST-16, with a 743 ms look-ahead, achieves over 9% relative improvement in beat F1-score compared to the lowest-latency configuration,[5] but at a delay that would be perceptible as a lack of synchronisation between audio and visual events.

The challenge, then, is to select components and configurations that keep the beat-tracking and reaction path's latency within a perceptually acceptable range, on the order of tens of milliseconds rather than hundreds, while maintaining sufficient accuracy in the extracted musical features. This is not a matter of optimising a single component but of understanding how latency accumulates across the entire processing chain and making informed trade-offs at each stage. The theoretical framework presented in this section, particularly the latency taxonomy and the distinction between causal and non-causal processing, provides the analytical tools needed to reason about these design decisions in the system implementation described in later chapters.

2.5 Deep Learning Architectures for Audio Analysis

The beat tracking and music analysis systems discussed in Section 2.2 rely on deep neural network architectures that have been adapted from other domains, principally computer vision and natural language processing, to the unique challenges of audio signals. This section provides a theoretical overview of the four principal architecture families used in modern music information retrieval: convolutional neural networks (CNNs), recurrent neural networks (RNNs), convolutional recurrent neural networks (CRNNs), and Transformer-based models. Understanding these building blocks is essential for the chapters that follow, as the system developed in this thesis employs a pipeline that combines several of them.

2.5.1 Convolutional Neural Networks

The connection between audio and convolutional neural networks begins with the spectrogram representation introduced in Section 2.1. Once audio has been converted into a mel spectrogram, the resulting two-dimensional time-frequency matrix can be treated as a single-channel image, making it possible to apply CNNs directly.[27] A CNN processes its input by sliding small learnable filters (also called kernels) across the input matrix, computing a dot product at each position to produce a feature map that highlights local patterns such as harmonic structures, onset transients, or percussive events. After each convolutional layer, a non-linear acti-

vation function (typically a rectified linear unit, or ReLU) is applied element-wise. Successive convolutional layers build a hierarchy of increasingly abstract features: early layers detect simple edges or frequency bands, while deeper layers combine these into higher-order representations such as timbral textures or rhythmic motifs.

Pooling layers follow convolution to reduce the spatial dimensions of the feature maps, providing a degree of translational invariance and lowering computational cost. Max pooling, which selects the maximum value within a local window, is the most common variant. An alternative to increasing the receptive field through pooling is the use of dilated (atrous) convolutions, which insert gaps between the filter elements so that each layer covers a wider span of the input without increasing the number of parameters. In MIR, convolutional architectures have been successfully applied to a wide range of tasks, including beat tracking, onset detection, chord recognition, instrument classification, and automatic music tagging.[27]

2.5.2 Recurrent Neural Networks and LSTMs

While CNNs excel at capturing local spectral and temporal patterns within their receptive field, audio is inherently sequential: the musical significance of a note, chord, or beat depends on the context established by preceding events. Recurrent neural networks (RNNs) address this by maintaining a hidden state vector that is updated at each time step using both the current input and the previous hidden state, creating a form of memory that allows information to propagate from earlier frames to later ones.[27]

In practice, standard RNNs struggle with long-range dependencies because gradients either vanish (shrink toward zero) or explode (grow unboundedly) during backpropagation through time. The Long Short-Term Memory (LSTM) architecture solves this problem by introducing a gating mechanism. Each LSTM unit contains three gates: an input gate that controls how much new information enters the cell state, a forget gate that controls how much of the existing cell state is retained, and an output gate that controls how much of the cell state is exposed as the unit's hidden output.[27] Together, these gates allow the LSTM to selectively remember or discard information over hundreds of time steps, making it far more effective than a standard RNN at modelling the kind of long-range temporal structure found in music, from phrase-level patterns to section-level repetitions.

Bidirectional variants (BiLSTMs) process the input sequence in both the forward and backward directions, concatenating the two hidden states at each time step to produce a representation that incorporates both past and future context. BiLSTMs are commonly used in offline MIR tasks where the entire recording is available before processing begins.[27]

2.5.3 Convolutional Recurrent Neural Networks

The complementary strengths of CNNs and RNNs lead naturally to their combination in convolutional recurrent neural networks (CRNNs). In a CRNN, a stack of convolutional layers first extracts compact, high-level feature vectors from the spectrogram input, and these feature sequences are then passed to one or more recurrent layers (typically LSTMs or GRUs) that model their temporal evolution. This design separates two distinct modelling responsibilities: the CNN front-end learns what to extract from each spectrogram frame (spectral shape, harmonic content, onset strength), while the RNN back-end learns how these features change over time (tempo regularity, metrical cycles, structural boundaries).[27]

The CRNN has become the dominant architecture for frame-level music analysis tasks, including beat tracking, onset detection, chord recognition, and vocal melody extraction.[27] A representative example discussed in Section 2.2 is BeatNet, whose architecture consists of three convolutional layers followed by four LSTM layers, producing frame-wise beat and downbeat activation functions from a log-frequency spectrogram input.[10] CRNNs have also been the standard architecture for online (causal) beat tracking systems, where the recurrent component naturally processes audio one frame at a time in a streaming fashion.[5] The practical success of the CRNN across so many MIR tasks can be attributed to this clean separation of concerns: convolution handles the local, spatial structure of the spectrogram, while recurrence handles the global, temporal structure of the music.

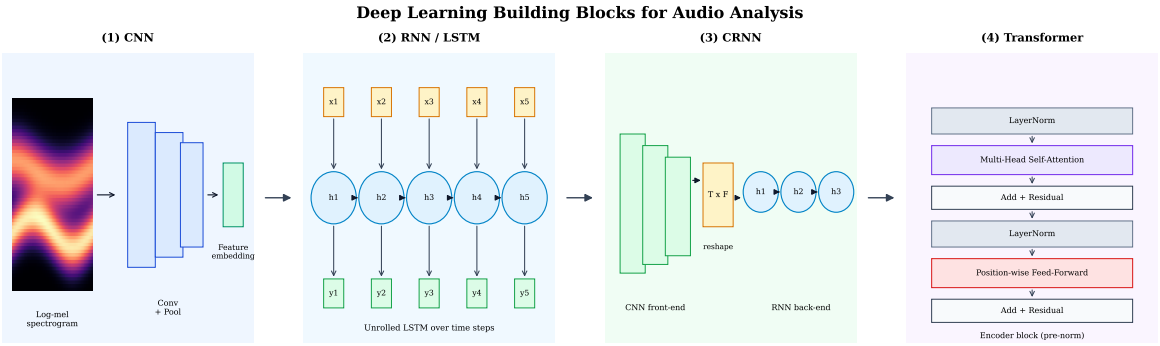

Figure 2.9: Deep learning building blocks for audio analysis, shown as increasingly complex architecture families: CNN, RNN/LSTM, CRNN (CNN + RNN), and Transformer (self-attention with feed-forward blocks). Arrows indicate progression and extension across model families. Adapted from Purwins et al. (2019).

2.5.4 Attention and the Transformer Architecture

Despite their effectiveness, recurrent networks process sequences one step at a time, creating a computational bottleneck and making it difficult to capture dependencies

between distant time steps efficiently. The attention mechanism addresses this limitation by allowing the network to compute a weighted combination of all positions in a sequence when producing the representation for any single position, effectively bypassing the sequential processing constraint.[27] Rather than compressing an entire history into a single fixed-size hidden state, attention lets the model look back at all previous (or all surrounding) time steps and assign a learned relevance weight to each, so that the most informative positions contribute most to the current output.

The Transformer architecture, introduced by Vaswani et al. (2017),[1] builds entirely on self-attention and dispenses with recurrence altogether. In each Transformer layer, multi-head self-attention computes several parallel attention functions, each learning to attend to different aspects of the input relationships (for instance, one head might attend to rhythmic periodicity while another captures harmonic similarity). This is followed by a position-wise feed-forward network, with residual connections and layer normalisation stabilising training. Because the Transformer has no inherent notion of sequential order, explicit positional encodings must be added to the input to provide information about the position of each element in the sequence. Since all positions are processed in parallel rather than sequentially, Transformers are both faster to train on modern GPU hardware and more effective at modelling long-range dependencies than RNNs, though their memory and computation scale quadratically with sequence length.

The adoption of Transformers in MIR is relatively recent but rapidly growing. BEAST (Chang and Su, 2024) was the first system to apply a streaming Transformer to online beat and downbeat tracking, replacing the recurrent layers of the traditional CRNN pipeline with a Transformer encoder that uses contextual block processing.[5] To handle the streaming (causal) constraint, BEAST employs relative positional encodings inspired by Transformer-XL, which allow the model to attend to a bounded window of past context without requiring access to the full recording.[5] At the offline end of the spectrum, the "Beat this!" system of Foscarin et al. (2024) combines a convolutional front-end with a Transformer backbone consisting of six encoder blocks using rotary positional embeddings, demonstrating that a convolution-plus-Transformer design can achieve state-of-the-art beat tracking accuracy without any recurrence-based post-processing.[8] These systems illustrate a broader pattern in which Transformers are not replacing CNNs entirely but rather replacing the recurrent component of CRNNs, yielding hybrid convolution-Transformer architectures that combine the local feature extraction of convolutions with the global context modelling of self-attention.

Beyond single-task classification, the Transformer architecture also supports encoder-decoder configurations for sequence-to-sequence tasks, where one sequence (such as a musical audio representation) is mapped to a different output sequence (such

as a lighting control sequence). The BART (Bidirectional and Auto-Regressive Transformers) architecture exemplifies this design: it combines a bidirectional Transformer encoder, which reads the entire input in full context, with an autoregressive Transformer decoder, which generates the output sequence one token at a time.[34] BART is pre-trained using a denoising objective in which the input is corrupted (for example, by masking spans of tokens) and the model learns to reconstruct the original, giving the encoder a deep understanding of the input structure.[34] This encoder-decoder formulation is central to the Skip-BART model used in this thesis and is examined in detail in the following section.

The progression from CNNs through CRNNs to Transformers reflects a broader trend in deep learning for audio: each successive architecture captures increasingly global temporal context while reducing or eliminating the sequential bottleneck inherent in recurrence. The system developed in this thesis draws on multiple points along this progression. The beat tracking component relies on CRNN and Transformer-based architectures as discussed in this section, while the lighting generation component uses the BART encoder-decoder architecture, whose sequence-to-sequence formulation is the subject of Section 2.6.

2.6 Sequence-to-Sequence Models and the BART Architecture

The previous section surveyed the deep learning building blocks used for audio analysis: CNNs for local spectral pattern extraction, RNNs for temporal modelling, CRNNs that combine both, and Transformers that replace recurrence with self-attention. The present section narrows the focus to one particular configuration of the Transformer that is central to the lighting generation component of this thesis: the encoder-decoder, or sequence-to-sequence, architecture, and its instantiation in BART.

2.6.1 The Sequence-to-Sequence Framework

Many audio processing tasks are essentially sequence-to-sequence transduction tasks, in which an input sequence of one modality or representation must be mapped to an output sequence of potentially different length or modality. A sequence-to-sequence model transduces an input sequence into an output sequence directly, without requiring the two sequences to be the same length or to share a common vocabulary. The paradigm emerged from neural machine translation, where the goal is to map a sentence in one language to a sentence in another, but it has since been adopted for speech recognition, music transcription, and, as this thesis demonstrates, cross-

modal generation tasks such as mapping music to lighting.

In the formulation described by Purwins et al. (2019), the context at each decoder step is a weighted sum of the encoder embeddings, where the weights are calculated between the current decoder embedding and all encoder embeddings, indicating correlations between input and output positions.[27] With the adoption of RNNs for sequence modelling, the research field shifted towards these full sequence-to-sequence models, and despite their architectural simplicity, further improvements in both model structure and optimisation process have been proposed.[27]

The Transformer architecture, discussed in Section 2.5, replaced the recurrent components of the encoder and decoder with self-attention layers, yielding a fully attention-based sequence-to-sequence model. This Transformer-based encoder-decoder design forms the backbone of the architecture discussed next.

2.6.2 Denoising Pre-training and the BART Architecture

BART (Bidirectional and Auto-Regressive Transformers), introduced by Lewis (2019),[11] combines a bidirectional Transformer encoder with an autoregressive Transformer decoder.[34] The encoder reads the entire input sequence with full bidirectional attention, building a rich contextual representation of every position. The decoder then generates the output sequence left-to-right, attending to the encoder's representations through cross-attention and to its own previously generated tokens through causal (masked) self-attention.

What distinguishes BART from encoder-only models such as BERT or decoder-only models such as GPT is that it is pre-trained using a denoising objective: the input text is corrupted by applying one or more noise functions (such as token masking, token deletion, sentence permutation, or span infilling), and the model is trained to reconstruct the original, uncorrupted text.[11] This pre-training strategy forces the encoder to develop a deep understanding of the input structure while training the decoder to generate fluent, coherent output, making BART particularly effective for generation tasks.

The BART architecture has been successfully adapted beyond natural language. PianoBART (Liang et al., 2024) applies the same framework to symbolic music, pre-training on large-scale piano MIDI data to learn latent musical representations and then fine-tuning on downstream tasks such as melody extraction and emotion classification.[34] As discussed below, the Skip-BART model developed by Zhao et al. (2025) extends this cross-domain adaptation further, from symbolic music to a cross-modal setting in which the encoder processes audio representations and the decoder generates lighting control sequences.[34]

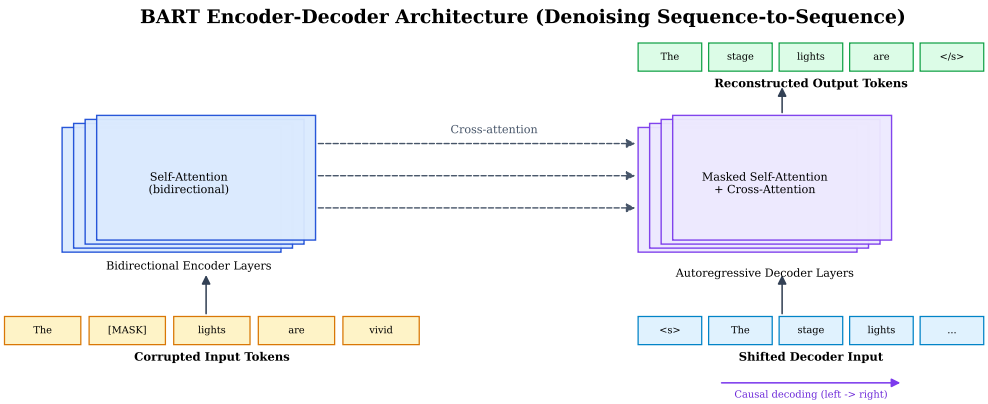

Figure 2.10: BART encoder-decoder architecture. The bidirectional encoder processes a corrupted input sequence, and the autoregressive decoder reconstructs the original sequence via masked self-attention and layer-wise cross-attention to encoder representations. Adapted from Lewis (2019) and the Skip-BART architecture description.

2.6.3 Cross-Modal Adaptation

The application of BART to automated stage lighting control (ASLC) requires bridging two fundamentally different modalities: continuous audio representations on the encoder side and discrete lighting parameters on the decoder side. Zhao et al. (2025) frame ASLC as a generative task rather than a rule-driven mapping process, arguing that lighting design is essentially an art creation process and proposing the first end-to-end deep learning method for stage lighting generation.[34][34]

On the encoder side, the music input is represented using OpenL3 (Cramer et al., 2019), a deep neural network architecture that extracts high-level auditory embeddings through audio-visual correspondence learning.[34] These continuous embeddings are passed through an MLP to align their dimensionality with the BART backbone before being fed into the encoder.[34]

On the decoder side, the lighting output is represented in the HSV colour space (discussed in detail in Section 2.3), with the Saturation channel fixed at maximum and only the Hue (H) and Value (V) channels modelled. Rather than treating these as continuous values, Skip-BART tokenises them into discrete categories: hue values range from 0 to 179 and value (brightness) levels from 0 to 255, yielding vocabulary sizes of 180 and 256 respectively.[34] This tokenisation approach is essential because directly processing hue data with an MLP presents challenges due to the cyclical nature of hue values: the minimum and maximum hue values can be quite similar, complicating the learning process; in contrast, the embedding layer effectively captures the relationships between different tokens, making it more suitable for handling cyclic data like hue.[34] The tokenised hue for each frame is computed as the mode of the hue distribution, while the tokenised value is computed as the weighted average of the value distribution.[34]

A distinctive architectural contribution of Skip-BART is its skip connection mechanism. When applying BART for the lighting generation task, the attention mechanism in the decoder struggles to determine which light frame corresponds to which music frame, even though the music input and light output have the same temporal length. To address this, the skip mechanism directly combines the music embedding and light embedding before inputting them to the decoder, explicitly indicating the one-to-one correspondence between music frames and light frames. This is particularly important because the model must learn this frame-level alignment from complex and noisy data in an end-to-end manner.[34]

The backbone of Skip-BART directly incorporates pre-trained parameters from PianoBART, providing a solid starting point for subsequent training through cross-modal transfer learning.[34] To combine knowledge from PianoBART's multiple downstream tasks (melody extraction, velocity prediction, composer classification, and emotion classification), the Drop And Rescale (DARE) method (Yu et al., 2024) is used to merge the fine-tuned parameters,[34] and LoRA (Low-Rank Adaptation; Hu et al., 2022) is employed for efficient tuning, keeping the majority of backbone parameters frozen.[34]

2.6.4 Parameter-Efficient Fine-Tuning and Inference

The combination of transfer learning, DARE merging, and LoRA results in a model with approximately 240 million total parameters but only 19 million trainable parameters.[34] This parameter efficiency is critical given the limited size of stage lighting datasets.

The model is pre-trained using a BART-based masked language modelling approach. During pre-training, only the music data is used (without light) to avoid information leakage. The encoder encodes the masked music sequence, while the decoder receives the shift-right ground truth as input and attempts to recover the original music sequence. Although there is a gap between the music modality in pre-training and the light modality in fine-tuning, this approach has been demonstrated to be effective, supported by the theory of cross-modal transfer learning.[34] During fine-tuning, the model is trained end-to-end to generate the corresponding lighting sequences using a next-token prediction objective framed as a classification task over the hue and value vocabularies.[34]

During inference, Skip-BART generates the lighting sequence in an autoregressive manner, producing one lighting token at a time conditioned on the full music input and all previously generated tokens.[34] To prevent model degradation while maintaining diversity, a stochastic temperature-controlled sampling method is employed, in which the temperature parameter controls the randomness of the sampling distribution.[34] Additionally, to avoid abrupt changes in the generated light-

ing, a restriction mechanism ensures that successive lighting tokens do not exceed a specified threshold in hue or value distance.[34] This combined approach is termed Restricted Stochastic Temperature-Controlled (RSTC) sampling: the probability of all categories exceeding the distance threshold is set to zero during sampling, ensuring both diversity and temporal smoothness in the generated output.[34]

The theoretical concepts introduced in this section, particularly the encoder-decoder architecture, the denoising pre-training paradigm, and the cross-modal adaptation strategies, provide the foundation for understanding how the system developed in this thesis generates lighting from audio input. With the theoretical foundations of audio analysis, beat tracking, music-visual mapping, and generative lighting now in place, the next chapter turns from theory to platform: it evaluates the technologies and deployment options for building this system as a real-time web application and derives the platform constraints that shape its design.

Chapter 3

Technology Selection and Platform Constraints

This chapter evaluates the technology, framework, and deployment decisions that define the platform for the prototype, rather than presenting the complete final system design. It is organised by system layer: rendering and 3D graphics, audio capture and processing, machine learning deployment, and frontend application architecture. For each layer, the selected tools are presented, alternatives are compared, and the rationale for selection is justified. The chapter concludes by deriving the platform constraints these decisions impose, which establish the design space developed methodologically and architecturally in Chapter 4.

3.1 The Web as a Platform for Real-Time Audio-Visual Applications

The decision to build this system as a web application is motivated by three considerations: accessibility, portability, and the maturity of the modern browser as an application runtime. A web application requires no installation, runs in any modern browser regardless of operating system, and can be shared with collaborators or evaluators by distributing a URL rather than a platform-specific binary. For a research prototype intended to demonstrate feasibility, this low barrier to entry is a meaningful advantage.

These benefits depend on the browser providing the capabilities required for real-time audio-visual processing. The modern browser exposes the necessary APIs: WebGL for hardware-accelerated 3D rendering, the Web Audio API for low-latency audio capture and spectral analysis, WebAssembly for near-native computational performance, and Web Workers for background-thread execution. Collectively, these APIs make interactive audio-visual applications viable within the browser sandbox.

Alternative approaches were considered. A native C++/OpenGL application would offer unrestricted GPU access and lower audio latency, but would require platform-specific builds and significantly more complex distribution. Electron inherits the browser's resource overhead while losing the zero-install advantage. Game engines such as Unity or Unreal Engine provide powerful rendering and audio capabilities, but introduce a heavy runtime and licensing considerations disproportionate for a stage-lighting research prototype. The web platform, complemented by a local server for ML inference (Section 3.4), provides the best balance of accessibility, development speed, and technical capability for this project.

3.2 3D Rendering: WebGL and Three.js

The visual core of this application is a real-time 3D rendering of a virtual stage with lighting fixtures. WebGL provides the browser's interface to the GPU: a JavaScript API based on OpenGL ES 2.0 that exposes a programmable rendering pipeline without plugins [31]. However, WebGL operates at a low level of abstraction, requiring manual management of vertex buffers, GLSL shader programs, and the rendering state machine, which makes it impractical for application-level development.

Three.js addresses this by providing a high-level, scene-graph-based abstraction over WebGL. As the dominant web 3D library [19, 20], it provides the building blocks this project requires: scene and camera management, a renderer, and a hierarchy of meshes, materials, and lights. Most relevant to this project is the SpotLight primitive, which exposes parameters for colour, intensity, cone angle, penumbra, distance, decay, and target direction. These map directly to the physical characteristics of stage lighting fixtures (PAR cans, moving heads), making SpotLight a natural abstraction for the virtual stage [31]. Three.js also supports shadow mapping, post-processing pipelines (bloom, colour grading), and fog simulation, providing the visual vocabulary needed for a convincing stage environment.

The principal alternative, Babylon.js, offers a more comprehensive feature set (built-in physics, visual editor, native WebXR) but produces significantly larger bundles, often exceeding 1 MB even with tree-shaking [2], compared to Three.js's approximately 155 KB gzipped [3]. Since this project requires only lighting, camera controls, and scene composition, Three.js's lighter footprint, larger community, and mature React integration (Section 3.5) make it the better fit.

WebGPU, a next-generation API superseding WebGL, has reached production readiness across all major browsers as of late 2025 [6, 9]. Three.js already supports a WebGPU renderer alongside its WebGL renderer, so choosing Three.js preserves a clear upgrade path without requiring architectural changes. This project targets the WebGL renderer for maximum compatibility.

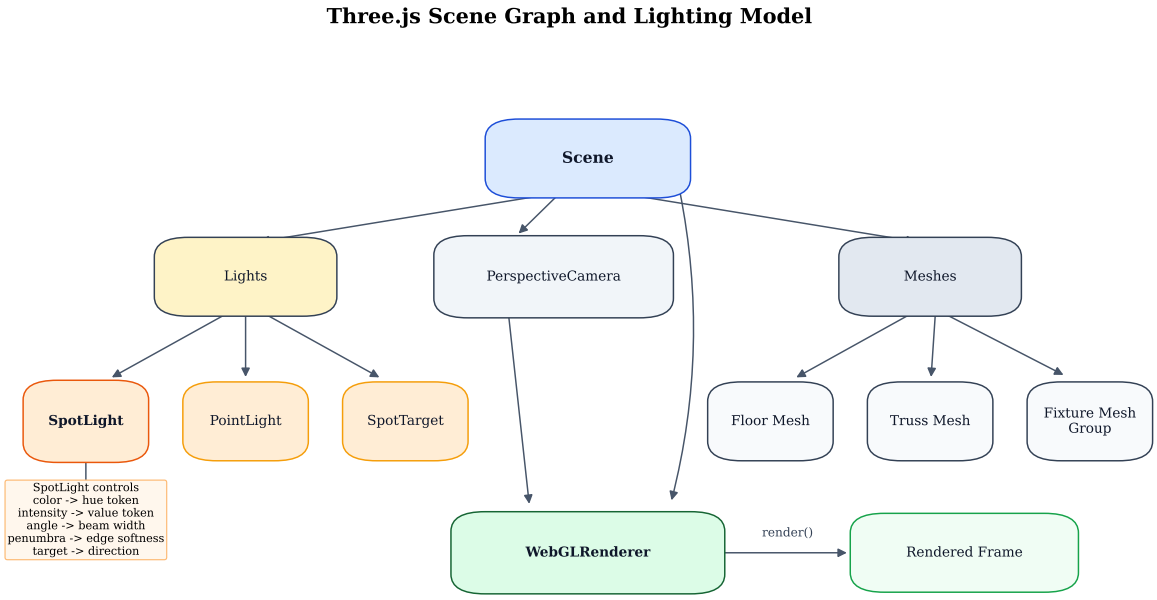

Figure 3.1: Three.js scene graph and lighting model used in this project. The diagram shows a `Scene` hierarchy with `PerspectiveCamera`, lighting nodes (`SpotLight`, `PointLight`, and `SpotTarget`), mesh groups for stage elements, and the `WebGLRenderer` consuming both scene and camera inputs to produce the rendered frame. SpotLight parameters correspond to stage-lighting controls: colour (hue), intensity (value), angle (beam width), penumbra (edge softness), and target (direction).

3.3 Audio Capture and Processing: The Web Audio API

The Web Audio API is a W3C specification that provides a graph-based audio processing architecture in the browser [33]. Unlike HTML elements, it exposes a modular system of audio nodes connected into a directed processing graph, enabling real-time analysis, transformation, and routing with low latency [17]. An AudioContext manages the graph and processes audio in render quanta of 128 samples (approximately 2.67 ms at 48 kHz) [14]. This application supports two audio input modes that produce an identical real-time stream from the perspective of downstream processing. For microphone input, `getUserMedia()` provides a MediaStream wrapped as a MediaStreamAudioSourceNode [16]. For file-based input, the uploaded audio is decoded into an AudioBuffer and played back through an AudioBufferSourceNode. Crucially, both modes feed the same processing graph frame by frame as audio plays, rather than pre-analysing the entire file. This unified streaming design ensures the system exercises identical real-time constraints regardless of source.

Two processing nodes are central to the pipeline. The AnalyserNode performs a real-time FFT on each audio block, exposing frequency-domain and time-domain data without modifying the audio stream [13]. It provides spectral data for the frontend visualisation and can supply initial frequency features to complement server-side analysis. The AudioWorklet runs a user-defined processor in a dedicated audio thread, separate from the main JavaScript thread, ensuring audio processing is not interrupted by UI activity [15]. In this project, the AudioWorklet extracts raw PCM audio chunks and transmits them to the server via WebSocket for consumption by the beat tracker and Skip-BART model.

The API imposes several design-relevant constraints. Browsers require a user gesture before an AudioContext can be activated (autoplay policy). The total audio output latency, the sum of `baseLatency` and `outputLatency`, typically ranges from 10 to 50 milliseconds on standard hardware [14]. Communication between the AudioWorklet thread and the main thread requires message passing via `port.postMessage()` or SharedArrayBuffers, since the AudioWorklet runs in an isolated scope.

3.4 Machine Learning Deployment

The system depends on two categories of ML models from Chapter 2: a beat tracker (BeatNet, BEAST, or PLP) for real-time rhythm analysis, and Skip-BART for generative lighting control. Both were developed in Python using PyTorch. Deploying them alongside a web application requires bridging the Python ML ecosystem and the browser runtime.

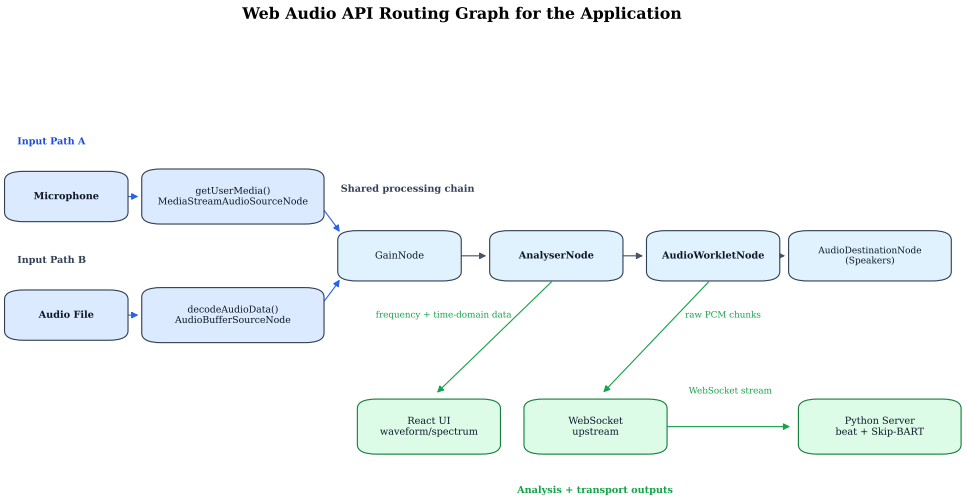

Figure 3.2: Representative Web Audio API routing configuration enabled by the selected browser stack. Microphone and audio-file inputs converge into a shared chain (`GainNode` → `AnalyserNode` → `AudioWorkletNode` → `AudioDestinationNode`), illustrating support for simultaneous playback, spectral analysis, and low-latency audio streaming to the inference backend.

3.4.1 Client-Side Inference Runtimes

Two runtimes support ML inference directly in the browser. ONNX Runtime Web runs models in ONNX format using WebAssembly (CPU) or WebGPU (GPU) backends, with PyTorch models exportable via `torch.onnx.export()` [21]. TensorFlow.js provides an alternative with WebGL, WebAssembly, and WebGPU backends, though models originating in PyTorch require a multi-step conversion path (PyTorch → ONNX → TensorFlow.js) [30]. However, client-side inference faces several constraints critical to this project. Skip-BART's 240 million parameters would require roughly 960 MB in FP32 (480 MB in FP16), approaching the 1 to 4 GB per-tab browser memory limits. Model conversion can fail with custom operators or dynamic control flow. The beat tracking models rely on Python-specific post-processing (Dynamic Bayesian Networks, particle filters, RSTC sampling) that would require reimplementation in JavaScript. Finally, browser GPU inference via WebGL/WebGPU remains less efficient than native CUDA.

3.4.2 Server-Side Inference

The alternative is running models natively in Python/PyTorch on a local server, which eliminates model conversion entirely. The original research implementations can execute with their published weights and post-processing logic, and a CUDA-capable GPU can be leveraged directly.

FastAPI is selected as the server framework because it provides native WebSocket support and an asynchronous architecture suitable for real-time streaming

[7]. This choice allows the Python model stack to remain unchanged while supporting a low-latency browser-to-server streaming loop.

3.4.3 Real-Time Communication

The hybrid architecture requires a persistent, bidirectional, low-overhead communication channel. The WebSocket protocol satisfies all three requirements: after an initial HTTP handshake, it maintains a full-duplex connection with minimal per-message framing overhead (2 to 14 bytes versus HTTP's 200 to 800 bytes of headers per request) [26]. On localhost, round-trip latency is sub-millisecond, making the network hop negligible relative to model inference time.

3.4.4 Justification for the Hybrid Architecture

Figure 3.3 summarises the deployment alternatives evaluated for this thesis. The decision to adopt server-side inference connected to the browser via WebSockets is driven by four considerations: (1) Skip-BART's 240 million parameters approach or exceed browser memory limits; (2) the Python-specific post-processing logic has no direct ONNX/TensorFlow.js equivalent, and reimplementing it in JavaScript would be a significant effort tangential to the thesis contribution; (3) native PyTorch with CUDA acceleration provides substantially faster inference, particularly for Skip-BART's autoregressive decoding; and (4) the clean separation into a rendering frontend and an analysis backend simplifies development and mirrors standard production ML architectures. The trade-off is an operational dependency on a running server, which is acceptable for a thesis demonstration context and is carried forward as a design constraint in Chapter 4.

3.5 Frontend Framework Selection: React and React Three Fiber

The frontend must support three concurrent concerns: rendering an interactive 3D stage, presenting control UI, and integrating high-frequency model outputs. The application is written in TypeScript for compile-time type safety across data contracts (audio features, WebSocket messages, and scene parameters) [32]. React provides the declarative component model for the UI layer (audio source selection, playback controls, fixture configuration). The 3D scene is implemented with React Three Fiber (R3F), a React renderer for Three.js that expresses Three.js objects as JSX components without abstracting away or adding overhead to the underlying library [24]. R3F's companion library, Drei [23] provides ready-made components directly applicable

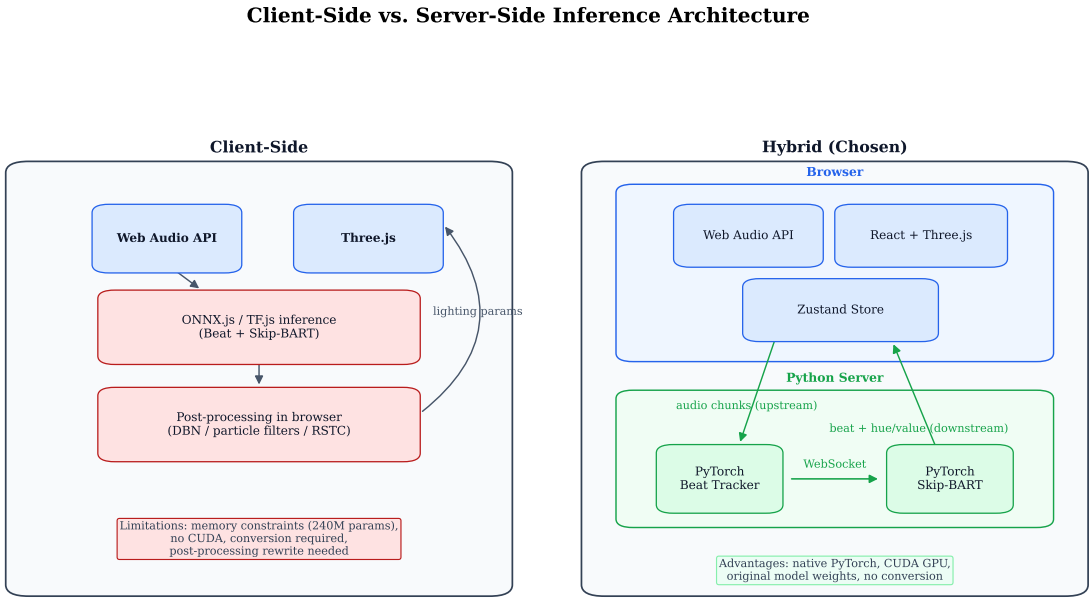

Figure 3.3: Client-side versus server-side inference architecture comparison. The left panel shows a pure browser pipeline (`Web Audio API`, in-browser model inference, post-processing, and Three.js rendering) and highlights its constraints: memory limits, lack of CUDA acceleration, model-conversion overhead, and post-processing reimplementation cost. The right panel shows the chosen hybrid architecture, where audio capture and rendering remain in the browser while native PyTorch beat tracking and Skip-BART inference run on a Python server, connected through bidirectional WebSockets.

to this project: OrbitControls for camera navigation, Environment for ambient lighting, and a SpotLight helper with volumetric cone visualisation.

State management uses Zustand, a subscription-based store from the same Poimandres collective [25]. Compared with heavier alternatives such as Redux-based setups, Zustand offers lower boilerplate and straightforward selective subscriptions, which is advantageous for real-time interfaces that mix UI concerns with rapidly changing visual parameters.

Alternative frameworks were evaluated. Vue with TresJS and Svelte with Threlte both offer declarative Three.js integration, with Svelte's compile-time approach eliminating framework runtime overhead. However, React Three Fiber's substantially larger ecosystem (companion libraries, community documentation, solved integration patterns) was the deciding factor for a thesis project where development speed is critical.

The selected React + R3F + Zustand stack therefore provides the required capability envelope for responsive UI control and high-frequency 3D updates in a browser setting. The detailed module responsibilities and runtime data-flow specification are presented in Chapter 4, where the full system design is developed.

3.6 Platform Constraints and Design Implications

The technologies presented in this chapter impose constraints that delimit the system design space.

JavaScript's single-threaded execution model means the main thread is shared between React rendering, DOM updates, and event handling. The AudioWorklet and Web Workers provide partial mitigation, but coordination still requires asynchronous message passing and explicit separation of UI work from time-sensitive audio/visual processing.

Browser memory limits (1 to 4 GB per tab) were a primary factor in adopting server-side ML inference. Skip-BART's 240 million parameters alongside the Three.js scene, React state, and audio buffers would leave insufficient headroom. WebGL's higher abstraction level compared to native APIs (no compute shaders, less efficient GPU memory management) is unlikely to be a bottleneck for this project's modest stage scene, but constrains future extensions. WebGPU addresses this prospectively (Section 3.2).

The Web Audio API's latency floor (10 to 50 ms total from `baseLatency` plus `outputLatency`) is additive with WebSocket round-trip time and server inference time. The perceptual thresholds established in Chapter 2 indicate that beat-driven visual reactions are perceived as synchronous when their end-to-end latency stays below approximately 100 ms, so the beat-synchronization path's budget must be managed across the entire chain. The generative lighting path is not bound by this threshold, since it refreshes the colour palette on a multi-second cadence rather than per beat. Cross-browser differences in WebGL extensions, AudioWorklet behaviour, and WebSocket handling led to targeting Chrome/Chromium as the primary runtime.

Finally, the hybrid architecture requires a running Python server, preventing standalone web deployment. For the thesis demonstration this is acceptable; containerisation (Docker) or cloud hosting could extend accessibility in future work.

These constraints define the system's design space. On that basis, Chapter 4 presents the methodological approach, derives the key requirements, and specifies the resulting system design in detail, including module boundaries, runtime data flow, and major design decisions.

Chapter 4

Methodological Approach, Requirements, and System Design

This chapter translates the technology and platform constraints from Chapter 3 into a concrete system design by defining the methodological framing, deriving the requirements the prototype must satisfy, and specifying the resulting architecture and runtime structure.

Positioned between technology selection (Chapter 3) and implementation (Chapter 5), this chapter serves as the design bridge between technical rationale and concrete execution.

4.1 Methodological Framing

The system is developed through an iterative prototype methodology oriented toward real-time interaction quality. Each iteration couples three activities: implementing a candidate processing and control path, measuring its responsiveness and stability during live playback, and refining module boundaries and communication patterns to reduce latency and improve robustness.

This methodology is constraint-driven rather than purely abstract. Architectural choices must respond to concrete limits already identified in the previous chapter, including browser thread contention, browser memory limits, real-time audio constraints, and the deployment implications of a hybrid browser-plus-Python stack.

This methodology follows from the problem itself: the system must analyse audio and generate lighting responses while music is playing. Because latency accumulates across audio capture, feature extraction, inference, message transport, and rendering, the design cannot optimize any single stage in isolation.

A further methodological consequence is that design quality must be judged not only by algorithmic correctness, but also by whether the integrated prototype

remains perceptually responsive during use. Chapter 2 established that sound-to-light systems tolerate more delay than instrumental interaction, but still require synchronization tight enough to appear aligned with musical events. For this reason, the present chapter treats system design as the disciplined management of latency, modularity, and usability under the constraints imposed by the chosen platform.

4.2 System Requirements

Based on the thesis objectives, the Chapter 3 platform constraints, and the earlier application requirements, the system design is guided by a compact set of functional and non-functional requirements.

4.2.1 R1. Real-time responsiveness

The system produces two kinds of visual output with different responsiveness needs, and R1 applies to each differently. On the beat-synchronization path, where detected beats drive immediate visual reactions, end-to-end audio-to-reaction latency must remain within a perceptually synchronous budget—a sub-100 ms target justified by the Chapter 2 discussion of sound-to-light latency and by the practical requirement specification. On the generative lighting path, where Skip-BART produces the colour palette, the requirement is instead one of cadence: the lighting must refresh often enough to track the music's evolution. Because Skip-BART generates lighting for a whole window at once rather than streaming, this path is not held to the sub-100 ms threshold, and its responsiveness is measured by update rate rather than per-frame latency.

4.2.2 R2. Stable browser interaction

User-interface controls and 3D rendering must remain responsive while audio analysis and model inference are active. This requirement follows from the single-threaded nature of browser execution and from the goal of presenting an interactive stage environment rather than a passive offline visualization.

4.2.3 R3. Native model fidelity

Beat tracking and generative lighting inference should run with native Python and PyTorch implementations rather than browser-converted approximations. This preserves compatibility with published model weights and post-processing logic, and avoids the conversion burden and performance penalties associated with an in-browser Skip-BART deployment.

4.2.4 R4. Deterministic streaming interfaces

Communication between browser and server must use explicit, low-overhead message contracts suitable for real-time streaming and debugging. The chosen communication mechanism must support persistent bidirectional transfer, since raw audio travels upstream while beat and lighting parameters travel downstream in real time.

4.2.5 R5. Dual audio-source support

The prototype should support both microphone capture and audio-file playback as first-class input modes. From the standpoint of downstream processing, both modes should be normalized into the same real-time processing path so that the system is evaluated under consistent operating conditions.

4.2.6 R6. Playback and monitoring controls

The interaction layer should expose the controls necessary to inspect and operate the system during a thesis demonstration, including audio upload or microphone input, playback controls, timeline navigation, and live feedback such as BPM or inference status. These functions are justified because the prototype is intended not merely to compute outputs, but to make the underlying MIR pipeline observable and operable in real time.

4.2.7 R7. Lighting and scene controls

The prototype should allow the user to influence the generated output through lighting and scene-management controls such as enabling or disabling Skip-BART, adjusting global intensity or smoothing, manually overriding fixtures when needed, and saving or loading stage configurations. These requirements are appropriate at the design level because they describe user-facing capabilities rather than low-level implementation choices, and they support the broader project goal of turning research models into a practical interactive application.

4.2.8 R8. Thesis-demonstration deployability

The system must run reproducibly on a developer workstation with a browser frontend and a local inference server. This requirement reflects the accepted trade-off of the hybrid architecture: standalone web deployment is sacrificed in exchange for native model execution and tractable development complexity within the scope of the thesis.

Taken together, these requirements avoid overcommitting the design to outdated assumptions such as mandatory browser-side model conversion. The earlier requirements document considered ONNX or TensorFlow.js deployment paths and local server fallback, but the thesis now justifiably treats server-side inference as the primary architectural decision rather than a backup option.

4.3 Architectural Design Decisions

4.3.1 Hybrid deployment as the governing decision

The central architectural decision is a browser–server partition. The justification for this partition is established in Section 3.4; here, the focus is on the design consequences of that partition. The remaining decisions in this section — module ownership, state management, and communication protocol — derive from this split rather than re-justify it.

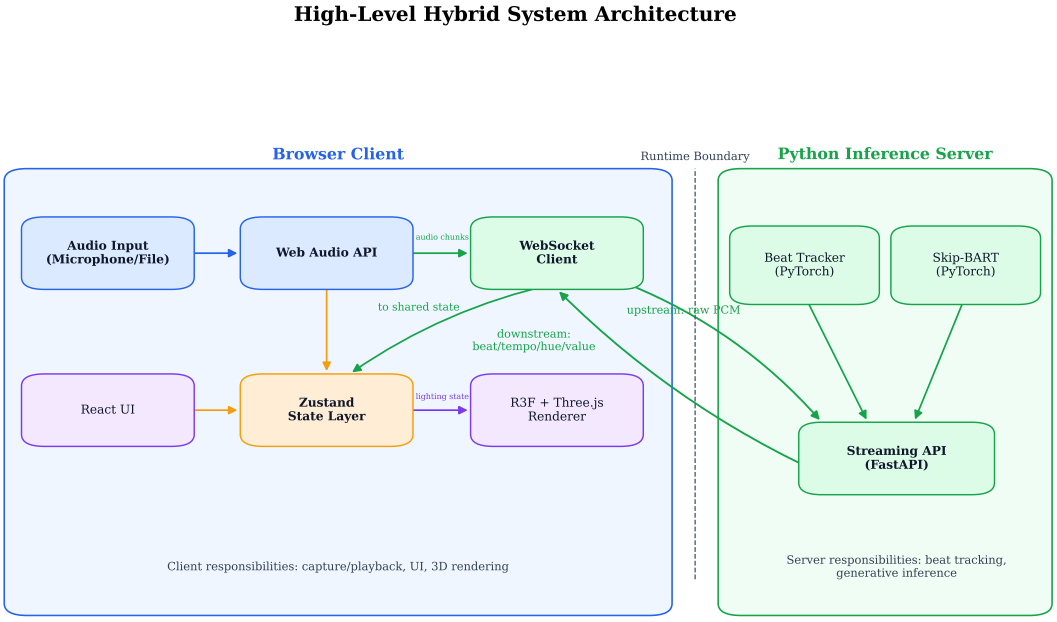

Figure 4.1: High-level hybrid system architecture. The browser client handles audio input, user interface, shared frontend state, and Three.js rendering, while a local Python inference server hosts beat tracking and Skip-BART. A bidirectional WebSocket channel connects the two runtimes for upstream audio streaming and downstream control outputs.

4.3.2 Browser-side audio handling and rendering

The browser owns user-proximal tasks: audio input, the Web Audio graph, local analysis, 3D rendering, and the user interface. This allocation preserves the acces-

sibility advantages of the web platform while ensuring that interaction and visualization remain tightly coupled to the presentation layer. The detailed treatment of how the two audio sources are unified into a single processing path is presented in Section 4.4.2 as part of the resulting frontend data flow.

4.3.3 Native inference on the Python server

The Python server module owns two responsibilities: real-time beat tracking and Skip-BART inference. Both run in their original Python and PyTorch form so that published model weights and post-processing logic remain authoritative, in line with Requirement R3.

The design implication is that the server is treated as the canonical environment for model execution. Any other module that needs inference outputs must obtain them through the streaming interface defined in Section 4.3.5, rather than maintaining a parallel inference path.

4.3.4 Shared state and selective rendering updates

To satisfy Requirement R2 under the browser execution model identified in Chapter 3, the architecture must avoid pushing high-frequency lighting updates through unnecessarily expensive UI reconciliation paths. The selected React + R3F + Zustand stack supports this by pairing a declarative UI layer with a 3D rendering layer and a lightweight shared store suited to rapidly changing visual parameters.

The resulting design rule is that the UI and the rendering loop consume shared state differently but coherently: UI components subscribe to the control and status slices they need, while the rendering path applies lighting updates at frame rate. The same store is the single source of truth for both interaction and rendering.

4.3.5 Communication protocol and runtime assumptions

The browser–server boundary uses the WebSocket transport selected in Section 3.4.3. At the design level, what matters is the message contract on that channel: raw PCM chunks travel upstream from the browser to the server, while beat positions, tempo values, and lighting parameters travel downstream. This direction of flow is what the rest of the system design relies on, and it is the integration point referenced by both Section 4.3.3 and Section 4.3.4.

The intended runtime is a developer workstation running a Chromium-based browser and a local Python server, in line with Requirement R8.

4.4 Resulting System Architecture

4.4.1 High-level module decomposition

The decisions above yield a hybrid client-server architecture with the following principal runtime modules: an audio-input and playback layer, a Web Audio processing layer, a transport layer, a shared frontend state layer, a user-interface layer, a Three.js rendering layer, and a backend inference layer. Each module has a focused responsibility while participating in a single continuous real-time pipeline.

4.4.2 Frontend runtime data flow

Within the browser, microphone and file-based inputs are normalized into a shared Web Audio processing graph. That graph simultaneously supports audio playback, local analysis through the AnalyserNode, and stream extraction through the AudioWorkletNode.

Downstream server messages carry beat positions, tempo updates, and lighting parameters. These outputs update shared frontend state, from which both the user-interface layer and the 3D rendering path consume synchronized values. UI controls also write user-adjustable parameters back into that state layer, ensuring that interaction and rendering operate over a single coherent model of the current system state.

4.4.3 End-to-end runtime pipeline

At runtime, the pipeline proceeds in five stages:

1. audio is captured from the microphone or decoded and replayed from a file in the browser;
2. the Web Audio graph provides local analysis support and exposes streamable audio chunks through the AudioWorklet;
3. those chunks are transmitted over WebSocket to the Python server;
4. the server performs beat tracking and Skip-BART inference and emits beat and lighting parameters;
5. the browser ingests these outputs into shared state and applies them to interface updates and scene rendering in real time.

This pipeline satisfies the architectural intent established earlier in the chapter. Rendering and interaction stay close to the user-facing runtime, whereas model execution remains in the environment best suited to native PyTorch workloads. The

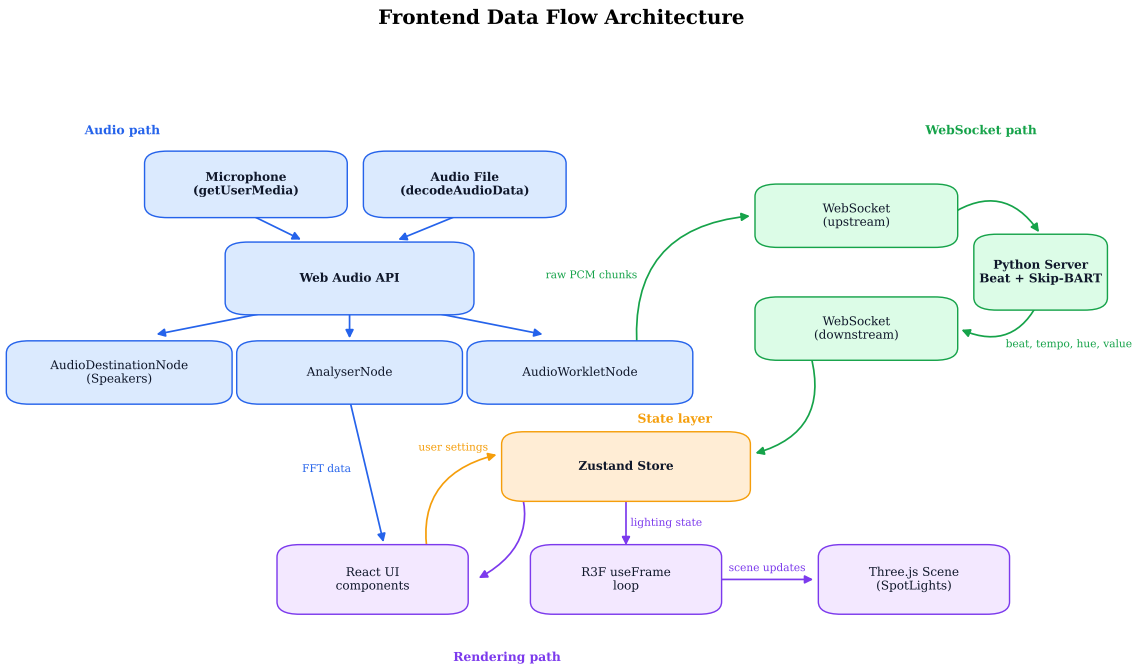

Figure 4.2: Frontend runtime data-flow architecture showing microphone and file inputs entering the Web Audio graph, branches to playback/analysis/streaming, bidirectional WebSocket exchange with the Python inference server, and shared frontend state consumed by React UI components and the R3F rendering path.

overall design therefore operationalizes the Chapter 3 technology selections while satisfying the methodological and requirement-level criteria defined above.

4.4.4 Latency-budget perspective and bridge to implementation

The architecture can also be read as an explicit allocation of the latency budget identified in Chapter 3. Each design decision earlier in this chapter has a budget consequence: native inference removes model-conversion overhead, the WebSocket channel removes per-message protocol cost, the shared-store rendering rule removes UI reconciliation cost on the hot path, and the unified Web Audio graph removes input-mode-specific divergence. Together, these decisions are what make the sub-100 ms end-to-end target for the beat-synchronization path structurally feasible rather than incidentally achieved. The generative lighting path falls outside this budget; its responsiveness is bounded by the periodic Skip-BART window pass rather than by the transport and rendering costs decomposed here (Chapter 6).

The design therefore commits to a latency-aware partition of responsibilities. Chapter 5 examines how that partition is realized in code, what engineering trade-offs arose during implementation, and what evidence demonstrates a working real-time prototype.

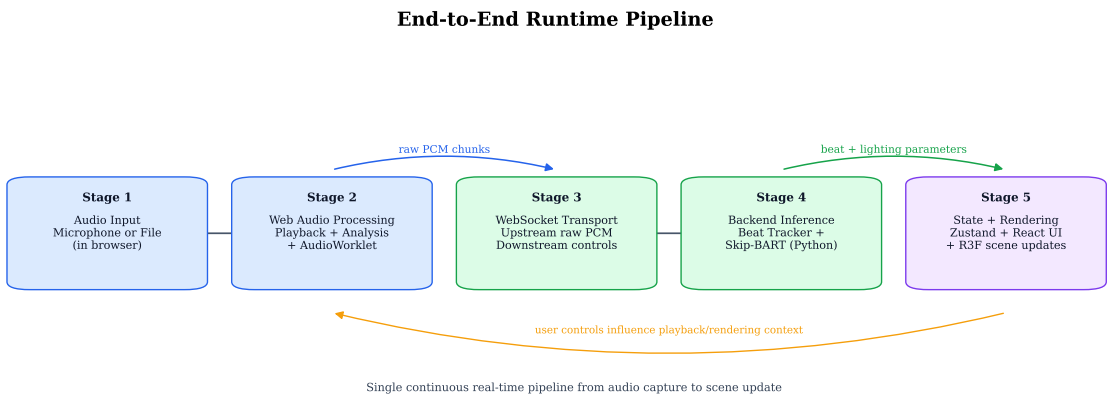

Figure 4.3: End-to-end runtime pipeline. The figure summarizes five stages: audio input, browser-side Web Audio processing, WebSocket transport, backend beat-tracking and Skip-BART inference, and final state-driven scene updates in the browser.

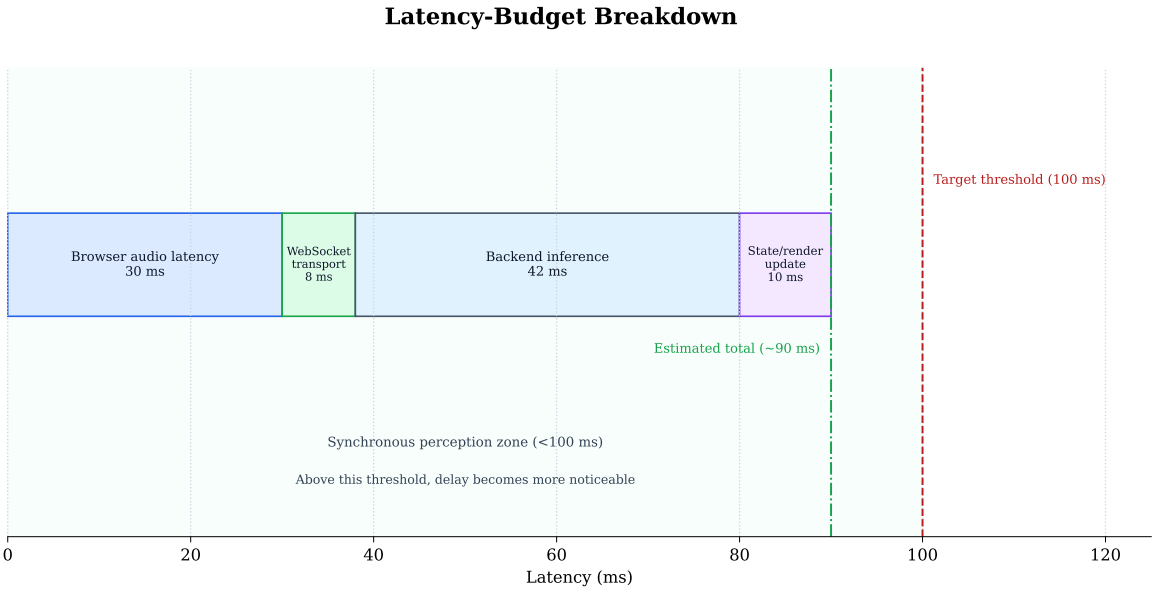

Figure 4.4: Latency-budget breakdown across the runtime pipeline. The chart decomposes estimated latency contributions from browser audio output, WebSocket transport, backend inference, and rendering/state update, and relates their aggregate to the sub-100 ms synchronization threshold for the beat-synchronization path.

Chapter 5

Implementation

Chapter 4 committed the system to a latency-aware browser–server partition and closed by promising to show how that partition is realised in code, what engineering trade-offs arose, and what evidence demonstrates a working prototype. This chapter delivers the first two; the evidence is the subject of Chapter 6. Rather than narrating every module, it is organised around the engineering problems that most shaped the prototype, each illustrated with a short extract from the codebase. The implementation was developed incrementally over roughly fifty commits, and several of the decisions below are best understood as responses to concrete failures encountered along the way.

5.1 Repository and Toolchain

The project is a single repository combining a TypeScript frontend and a Python backend. The frontend (`apps/web`) is a Vite, React, and React-Three-Fiber application responsible for audio capture, the 3D stage, and the user interface. The backend (`services/inference`) is a FastAPI WebSocket server that hosts BeatNet and Skip-BART under a conda environment with PyTorch. Two shared packages sit between them: `packages/protocol` defines the WebSocket message contract, and `packages/fixtures` defines the scene-document schema and fixture catalogue. This layout keeps the three concerns that must agree across the runtime boundary—protocol, scene model, and shared types—in one place, version-controlled together.

5.2 The Protocol as a Typed Contract

Requirement R4 calls for explicit, low-overhead message contracts. The implementation enforces this by making the protocol package the single authority for every message that crosses the WebSocket, defined once as a set of Zod schemas and mir-

rored by Pydantic models on the backend. Each message is a member of a discriminated union keyed on a `type` field, so an unknown or malformed payload is rejected at the boundary rather than fault deep in the pipeline.

Listing 5.1: Downstream lighting message, defined once in the protocol package (TypeScript/Zod). The backend mirrors it in Pydantic.

```
export const lightingUpdateSchema = envelopeSchema.extend({
  type: z.literal('lighting.update'),
  hue: z.number().min(0).max(360),
  value: z.number().min(0).max(1),
  intensity: z.number().min(0).max(1).nullable(),
  // client clock of the chunk that produced this frame -> e2e latency
  originChunkTimestampMs: z.number().optional(),
  serverProcessingMs: z.number().optional(),
});
export type LightingUpdate = z.infer<typeof lightingUpdateSchema>;
```

The two diagnostic fields in Listing 5.1 are optional, which is what allowed the benchmark instrumentation of Chapter 6 to be added later without breaking compatibility with messages that predate it. The same envelope carries a protocol version, so a frontend and backend that disagree fail loudly at session initialisation instead of silently misinterpreting fields.

5.3 One Audio Path for File and Microphone

Requirement R5 asks that file playback and microphone capture be first-class input modes, and the architecture requires them to converge before transport so that downstream processing is source-independent. Both sources feed the same Web Audio graph and are encoded into the same canonical chunk—mono, 48 kHz, Float32 PCM, base64-encoded, and sequenced—before being sent upstream. The transport layer never learns which source produced a chunk.

One consequence of sharing a single audio graph surfaced as an audible bug: in microphone mode the speakers fed back into the input. The fix routes monitoring through an explicit gain stage that file playback re-enables and microphone capture mutes, visible in the file player's `play()` path.

Listing 5.2: Restoring speaker monitoring on file playback; microphone mode mutes the same stage to prevent feedback.

```
graph.setSource(node);
// Restore speaker output: mic mode would have muted it.
graph.setMonitor(true);
```

5.4 Streaming a Model That Was Not Built to Stream

The most significant implementation challenge is that Skip-BART is not a streaming model. As discussed in Chapter 2, it is autoregressive over a fixed-length input sequence and was designed to generate a lighting track for a whole piece of music in a single offline pass. The prototype, by contrast, receives audio in small chunks while the music plays. Bridging this gap is the core of the integration work, and it is done with three mechanisms.

First, the backend maintains a rolling raw-audio buffer of roughly the last ten seconds and re-runs inference over that window periodically rather than continuously. The window is bounded to keep each pass tractable, and a frame cursor ensures that only lighting frames newer than the last emitted one are published—without it, the restricted stochastic sampling would re-emit overlapping frames with different sampled colours on every pass, producing visible flicker.

Listing 5.3: Sliding-window gate in the Skip-BART adapter: trim the buffer, advance the emit cursor, and only run inference once enough new audio has accumulated.

```
max_samples = int(WINDOW_SECONDS * self._buffer_sample_rate)
if self._buffer.size > max_samples:
    drop = self._buffer.size - max_samples
    self._buffer = self._buffer[drop:]
    self._buffer_start_ms += drop * 1000.0 / self._buffer_sample_rate
    if self._last_emitted_ms < self._buffer_start_ms:
        self._last_emitted_ms = self._buffer_start_ms

self._chunks_since_run += 1
if self._chunks_since_run < MIN_CHUNKS_BETWEEN_RUNS:
    return []           # not enough new audio yet
self._chunks_since_run = 0
```

Second, because a single window pass can take several seconds (Chapter 6 measures it at roughly eight on both test machines), running it inline would stall the chunk pipeline and freeze beat tracking. Inference therefore runs on a dedicated background worker thread: `ingest()` pushes a chunk onto a queue and returns immediately, the worker drains the queue and runs inference when the window is ready, and `get_predictions()` non-blockingly collects whatever the worker has produced. A monotonically increasing *generation* counter, checked both before processing a chunk and before publishing each predicted frame, lets a mid-stream session reset discard in-flight work without interrupting an inference already under way.

Listing 5.4: Worker-thread ingest. The generation guard discards chunks enqueued before a session reset.

```
def _worker_loop(self) -> None:
    while not self._stop_event.is_set():
        try:
            gen, chunk = self._in_queue.get(timeout=0.5)
        except queue.Empty:
            continue
        with self._gen_lock:
            if gen != self._generation:
                continue   # reset happened; drop stale chunk
        for pred in self._process_chunk(chunk):
            self._out_queue.put(pred)
```

Third, an early version loaded the models lazily on the first audio chunk, which produced a multi-second stall the first time playback started. Eager-loading the adapters at server startup moved that cost to launch time, where it does not affect interaction. The practical outcome of all three mechanisms is that lighting updates arrive in bursts—one burst per completed window pass—while the rest of the pipeline stays responsive. This burst behaviour is exactly what the lighting-rate measurements in Chapter 6 reflect, and it is a property of the integration strategy rather than of the transport or the worker design.

5.5 Keeping the Beat Path Real-Time

Unlike the generative lighting path, the beat-synchronization path must stay tight to the audio, so BeatNet runs on the synchronous chunk path and was reworked to process audio per analysis hop in a streaming fashion. A second problem emerged on real music: BeatNet misses beats, and on an otherwise steady track a dropped beat visibly stalls the fixtures. The pipeline compensates with a tempo-locked phase predictor that synthesises a beat when a predicted beat time passes without a real one, while a minimum-gap filter prevents a real beat and a synthesised beat from firing within the same musical pulse. Real beats always re-anchor the predictor's phase, so synthesis fills gaps without drifting.

Listing 5.5: Tempo-locked synthesis: emit a predicted beat once its time has passed, gated so it cannot land on top of a recent real beat.

```
while (
    self._next_predicted_beat_ms is not None
    and self._beat_interval_ms is not None
    and self._next_predicted_beat_ms <= chunk.timestamp_ms
```

```
):
    t = self._next_predicted_beat_ms
    if t - self._last_emitted_beat_ms >= min_gap:
        envelopes.append(beat_update(beat_time_ms=t, synthetic=True))
        self._last_emitted_beat_ms = t
    self._next_predicted_beat_ms += self._beat_interval_ms
```

The `synthetic` flag travels in the beat message (Listing 5.1 shows the analogous field on lighting), so the frontend and the benchmark can distinguish real from synthesised beats; the latter counted 110–120 of roughly 130 beats per run in Chapter 6, a direct measure of how often synthesis was doing the work.

5.6 Supporting Concerns

5.6.1 Scene state and the runtime mapping layer

The architecture keeps scene-document state and runtime lighting state separate but compatible. Every editor action mutates a `SceneDocument`, which the renderer draws from and which undo/redo and persistence operate on; the live lighting frame is never written into that document. Instead, a small pure function maps one lighting frame plus one fixture to the render state the 3D components consume, which is the only point at which runtime values touch the scene.

Listing 5.6: The single mapping point from runtime lighting to fixture render state (TypeScript). An idle floor keeps enabled fixtures visible before the first prediction.

```
const value = lighting.value > 0 ? lighting.value : IDLE_VALUE_FLOOR;
const intensity = instance.enabled ? baseIntensity * value : 0;
const hue = (((lighting.hue % 360) + 360) % 360) / 360;
out.setHSL(hue, SATURATION, LIGHTNESS);
```

Keeping this mapping pure and isolated means the rendering loop can apply high-frequency lighting updates without routing them through React's reconciliation, satisfying the responsiveness rule from Section 4.3.4.

5.6.2 Robustness and observability

Operating the prototype during development surfaced a class of failures that are invisible in offline code but routine in a live socket. Client disconnects mid-message were initially treated as errors and triggered an attempt to write to an already-closed socket; they are now treated as a clean exit. Backend stages are wrapped in lightweight timers feeding a rolling-window metrics registry—the same instru-

mentation later reused for the benchmark in Chapter 6—and a structured debug log surfaces WebSocket close codes in the interface. One non-obvious transport issue is worth recording: the protocol-level WebSocket keepalive had to be disabled in development because the Vite dev proxy drops control frames, which otherwise closed the connection spuriously.

5.7 The Application in Use

Figure 5.1 shows the running prototype. The central viewport renders the 3D stage—a truss structure carrying lighting fixtures over a reflective floor—driven live by the model output. The left panel is the fixture and structure catalogue used to compose the stage; the top bar holds audio-source selection, transport controls, and the stage import/export actions; and the right panel exposes the scene editor together with live model output and a debug log. The arrangement maps directly onto the module decomposition of Chapter 4: input and transport along the top, scene composition on the left, rendering in the centre, and shared state and diagnostics on the right.

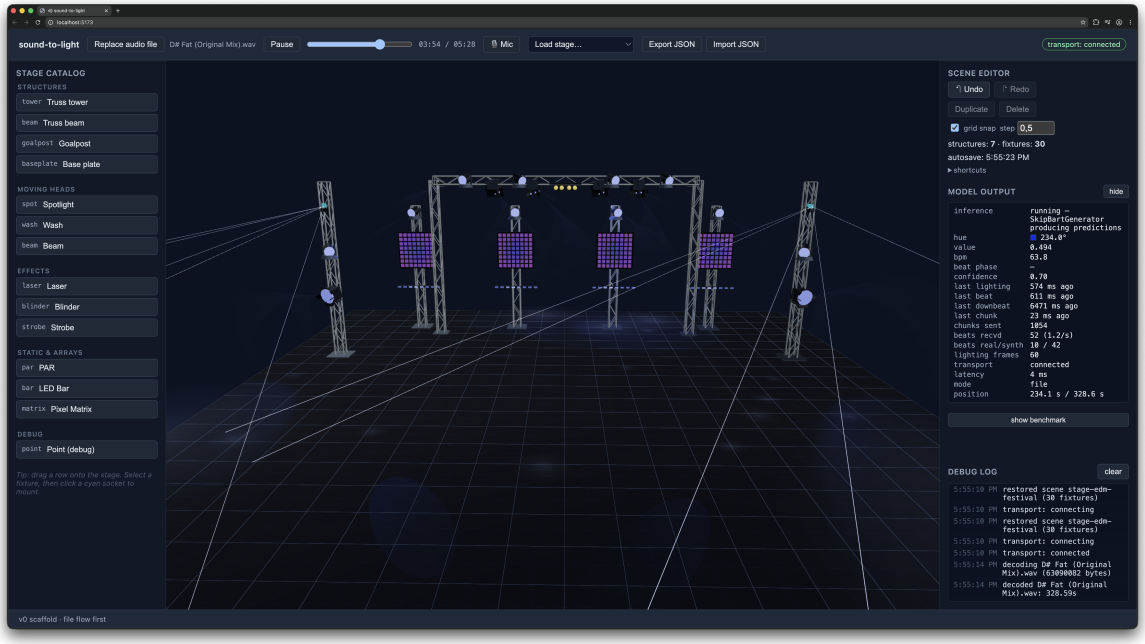

Figure 5.1: The sound-to-light prototype in use. The 3D stage (centre) is driven by live beat and lighting output from the inference server; the fixture catalogue (left), transport bar (top), and scene-editor, model-output, and debug panels (right) provide the controls and observability called for by requirements R6 and R7.

Taken together, the implementation realises the Chapter 4 partition while absorbing the practical cost of running research models in a live setting: a typed protocol at the boundary, a unified audio path, an offline lighting model adapted to stream through windowing and a worker thread, a beat-synchronization path with synthe-

sis, and a clean separation between scene state and runtime rendering. How well the resulting system performs, and where its limits lie, is quantified in Chapter 6.

Chapter 6

Evaluation

This chapter evaluates the runtime behaviour of the prototype and compares two deployment configurations of the same system: the inference backend running locally on the development laptop using Apple's Metal Performance Shaders (MPS), and the backend running on a desktop workstation with an NVIDIA CUDA GPU, accessed by the browser frontend over a local-area network. The goal is not to benchmark the underlying models against published baselines—BeatNet and Skip-BART are used as released—but to characterise how the integrated system performs under real-time streaming, where the bottlenecks lie, and how sensitive end-to-end behaviour is to the choice of compute device and network path.

The system under test is identical across configurations—same frontend, protocol, model weights, and audio source—so only the inference device and the transport path differ. A consequence is that two variables change at once between configurations: the compute device *and* the network path. The analysis therefore separates *server-side compute*, which isolates the device, from *end-to-end latency*, which additionally reflects transport; conflating them would wrongly attribute network cost to the device. This distinction is maintained in every figure and table.

6.1 Experimental Setup

6.1.1 Test configurations

Measurements were collected on the two machines summarised in Table 6.1.

Both machines were wired to the same router through a switch; no measurement used a wireless link. The Windows backend exposed its WebSocket endpoint through an SSH tunnel, which is the same mechanism used during development for remote-GPU operation.

Table 6.1: Test configurations. The Mac configuration runs the backend on the same host as the browser (loopback transport); the Windows configuration runs the backend on a separate desktop reached over a wired local-area network.

	Mac-local-MPS	**Windows-CUDA-LAN**
Role	Laptop (dev machine)	Desktop workstation
CPU	Apple M1 Pro, 10-core (8P+2E)	Intel i7-8086K, 6c/12t @ 4.0 GHz
GPU / backend	M1 Pro 16-core, MPS	NVIDIA RTX 2070 Super 8 GB, CUDA 12.4
Memory	16 GB unified	16 GB DDR4
Operating system	macOS 26.5	Windows 11 25H2 (build 26200.8457)
PyTorch	2.11.0	CUDA 12.4 build
Transport path	Loopback (same host)	Gigabit Ethernet + SSH tunnel

6.1.2 Workload and protocol

The audio source for every run was a single electronic dance music (EDM) track, *D# Fat (Original Mix)* (approximately 5 min 28 s, decoded to mono 48 kHz). The track was chosen deliberately: its strongly periodic, clearly articulated beat provides a stable rhythmic signal for the beat tracker, and—because Skip-BART was not trained on EDM—it represents an honest out-of-distribution input for the lighting model rather than a favourable one. All runs streamed audio from the same fixed start position to keep the workload identical.

Each configuration was measured three times. A run streamed audio for 65 s of wall-clock time, of which the first 5 s are treated as warm-up and excluded from client-side statistics, leaving roughly 60 s of measured streaming. Audio is transported as canonical 2048-sample chunks at 48 kHz (≈1,520 chunks per run); chunk accounting confirmed that the count received by the backend matched the count sent in every run, so no upstream samples were dropped.

6.1.3 Metrics

The benchmark records two families of measurements. *End-to-end* metrics are timed on the client as the interval between the timestamp of the audio chunk that triggered an update and the moment that update is applied in the browser; these include beat and lighting delivery latency and the WebSocket round-trip time (RTT). *Server-side* metrics are timed on the backend and exclude all transport: the per-chunk processing dwell, BeatNet's enqueue and inference-drain times, the Skip-BART window pass (OpenL3 feature extraction followed by autoregressive decoding), and the worker-thread queue handoffs that bound the asynchronous design's overhead.

Because Skip-BART is autoregressive and was not designed for streaming, the backend approximates streaming by periodically re-running inference over a rolling audio window on a background worker thread (Chapter 5). One inference sample

therefore corresponds to one window pass, which emits a *burst* of lighting frames rather than a single frame—a distinction central to interpreting the lighting results below.

6.2 Results

6.2.1 Lighting generation is feature-extraction bound, not device bound

The dominant cost in the pipeline is the Skip-BART window pass. Figure 6.1 shows its distribution pooled across the three runs of each configuration. The two distributions overlap almost entirely: the CUDA workstation averages 7.4 s per pass against 8.1 s on MPS, a difference of roughly 8 % that is small relative to the within-configuration spread (both range from about 2.2 s to over 11 s). A discrete GPU with native CUDA does not deliver the order-of-magnitude speed-up one might expect from the decode step alone.

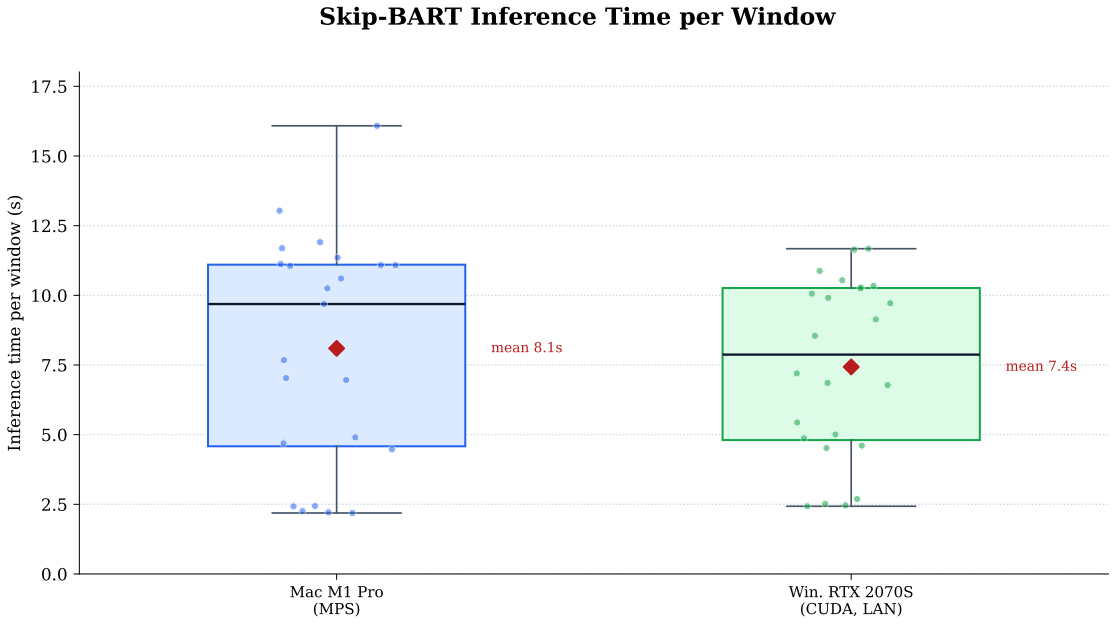

Figure 6.1: Skip-BART inference time per rolling-window pass, pooled across three runs per configuration. Boxes show quartiles, points show individual passes, and the diamond marks the mean. The heavy overlap indicates that the compute device is not the primary determinant of inference cost.

The explanation is that a `server.skip.inference` sample combines two stages with very different hardware profiles: OpenL3 audio-embedding extraction and the Skip-BART autoregressive decode. The embedding stage is the heavier and less GPU-amenable of the two in this deployment, so it dominates the combined time and compresses the gap between MPS and CUDA. The practical consequence is that

the effective lighting update rate is essentially the same on both machines—an average of approximately 2.5 lighting frames per second on MPS and 2.7 on CUDA—because the cadence is gated by this window pass rather than by transport or by the decode device. These frames are not delivered at a steady rate but in bursts, one burst per completed window pass: a fresh burst of colour therefore arrives only about once every 8 s on MPS and 7.4 s on CUDA, with each burst carrying roughly twenty frames. The 2.5–2.7 Hz figure is thus an average frame rate, not a burst rate; the perceived cadence of new lighting decisions is the multi-second window pass.

6.2.2 Server-side compute is comparable; BeatNet drain favours the M1 Pro

Figure 6.2 compares the remaining server-side stages, which are three to four orders of magnitude cheaper than the Skip-BART pass and therefore shown separately on a logarithmic axis. The worker-thread queue handoffs (skip enqueue/dequeue) are negligible at approximately 0.01 ms on both machines, confirming that the asynchronous design adds no meaningful overhead and that Skip-BART's cost is genuinely in computation, not in the surrounding plumbing.

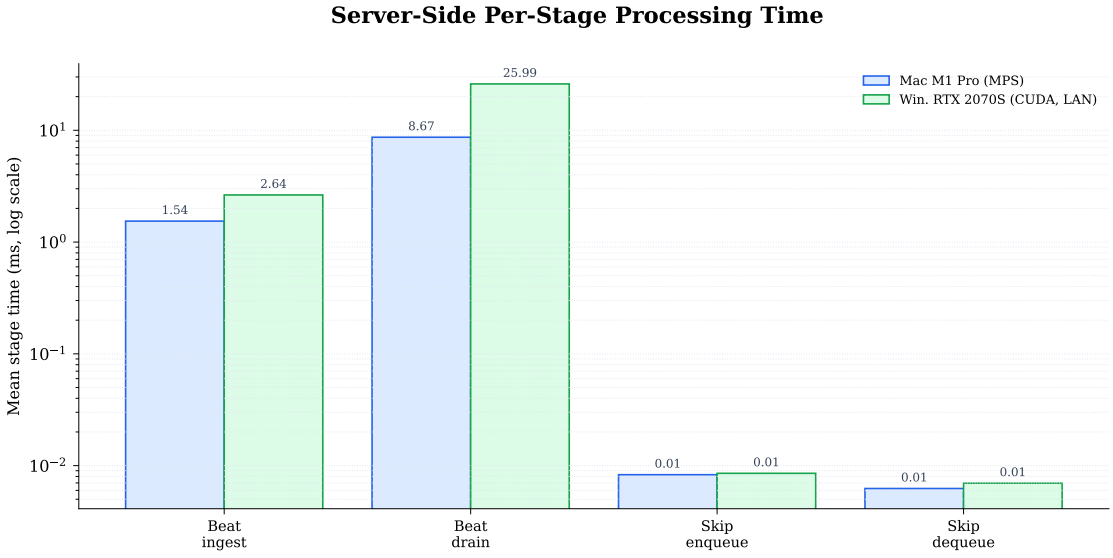

Figure 6.2: Mean server-side per-stage processing time (log scale), excluding the network path. Queue handoffs are negligible on both machines; BeatNet drain is the only stage with a clear device difference, favouring the M1 Pro.

The one stage with a clear difference is BeatNet's inference drain: a mean of 8.7 ms (95th percentile 59 ms) on the M1 Pro against 26.0 ms (95th percentile 181 ms) on the i7-8086K workstation. BeatNet's real-time path is sensitive to single-thread CPU performance and scheduling jitter, and the newer M1 Pro both runs it faster and with markedly less tail variance than the 2018-era desktop CPU. This prop-

agates into the server dwell time, whose mean rises from 13.0 ms on the Mac to 46.7 ms on the workstation while the medians remain close (1.5 ms versus 2.8 ms)—a tail-driven difference, not a shift in typical-case cost.

6.2.3 End-to-end latency is dominated by the network path

Figure 6.3 shows the cumulative distributions of beat and lighting delivery latency. For beat updates the separation is large: the median beat latency is 8 ms on the loopback Mac configuration but 102 ms over the LAN, and the tails diverge further still. On the Mac configuration the distribution is right-skewed: the median of 8 ms sits well below the mean of 23.8 ms and a 95th percentile of 100 ms (Table 6.2), the latter reaching the sub-100 ms synchronization threshold itself. This is a transport effect, not a compute effect—it tracks the round-trip time, which rises from a median of 3 ms (loopback) to 42 ms (LAN), with a 95th percentile of 715 ms caused by jitter introduced by the TCP/SSH tunnel rather than by the inference device.

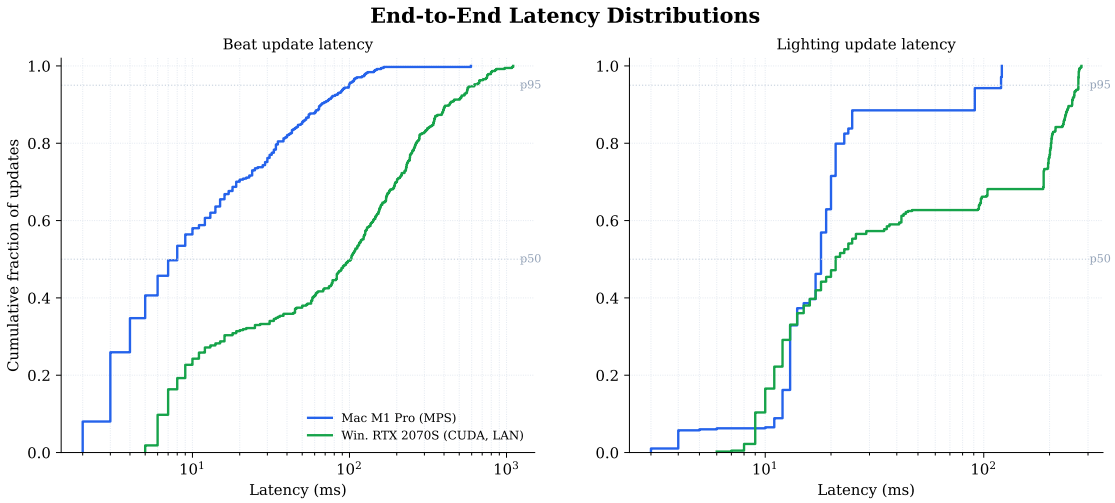

Figure 6.3: Cumulative distribution of beat (left) and lighting (right) end-to-end delivery latency, pooled across runs. The loopback Mac curves rise steeply; the LAN Windows curves are shifted right with long tails, reflecting transport cost rather than the compute device.

Lighting delivery latency is less separated than beat latency—medians of 18 ms and 21 ms respectively—because lighting frames arrive in bursts produced by a completed inference pass, so their delivery timing is shaped more by the burst structure than by per-message transport. The aggregate figures for all metrics, reporting median, mean, and 95th percentile where available, are collected in Table 6.2, kept in the two groups motivated above.

Table 6.2: Aggregated metrics, pooled across three runs per configuration. Server-side metrics isolate the compute device; end-to-end and network metrics additionally reflect the transport path (loopback for Mac, LAN for Windows). Lower is better throughout except for the lighting frame rate. The lighting frame rate is an average; frames are delivered in bursts of roughly twenty, one burst per completed window pass. Medians are omitted (—) for server-side metrics where they were not aggregated across runs.

	Mac-local-MPS			**Windows-CUDA-LAN**		
Metric	med.	mean	p95	med.	mean	p95
Server-side compute (device)						
Skip-BART inference (s)	—	8.10	13.04	—	7.43	11.64
BeatNet drain (ms)	—	8.67	58.74	—	25.99	180.84
BeatNet ingest (ms)	—	1.54	3.63	—	2.64	6.06
Skip enqueue/dequeue (ms)	—	0.01	0.01	—	0.01	0.02
Server dwell (ms)	1.5	12.98	74.54	2.8	46.67	247.60
End-to-end / network (device + transport)						
WebSocket RTT (ms)	3	14.08	83.00	42	125.67	715.00
Beat delivery (ms)	8	23.81	100.00	102	165.82	625.00
Lighting delivery (ms)	18	26.46	120.00	21	88.02	271.00
Lighting frame rate (Hz, avg)		≈2.50			≈2.69	

6.3 Discussion

Three conclusions follow from these measurements. First, the **beat-synchronization path meets real-time expectations** when the network is local: a median beat-to-light latency of 8 ms on the Mac configuration (mean 23.8 ms, 95th percentile 100 ms; Table 6.2) is well within the sub-100 ms window in which audio and visuals are perceived as synchronous for typical beats (Chapter 4), though the tail of the distribution reaches the threshold itself. The beat-synchronization path is the part of the system that most needs to be tight, and it is.

Second, the **lighting path is inference-bound, and that bound is largely device-independent**. Because a window pass costs several seconds and is dominated by feature extraction rather than by the GPU decode, switching from MPS to a CUDA GPU does not materially improve the lighting update rate. This reframes the system's principal performance limitation: it is not the WebSocket transport, the serialization overhead, or the worker-thread architecture—all of which are negligible—but the cost of repeatedly extracting embeddings over a rolling window for a model that was never designed to stream. Meaningful improvement would come from optimising or replacing the embedding stage, caching overlapping window computation, or adopting a streaming-native lighting model, rather than from faster decode hardware alone.

Third, the **end-to-end comparison is confounded by the transport path** and

must be read accordingly. The Mac configuration is loopback and the Windows configuration is a real LAN reached through an SSH tunnel, so the larger Windows latencies—and especially the heavy RTT tail—reflect transport, not the RTX 2070 Super. The fair device comparison is the server-side group of Table 6.2, where the two machines are close and the M1 Pro's only clear advantage is in BeatNet drain consistency.

These results should be read with their limitations in mind. They cover a single audio track on two machines with three runs each; the SSH tunnel adds latency and jitter that a production deployment on the same subnet could reduce; and the headline cost—the Skip-BART window pass—was measured as a single combined stage, so the precise split between embedding and decode is inferred rather than directly instrumented. A finer-grained breakdown of the inference stage would be a natural next step.

Chapter 7

Conclusions and Future Work

This thesis set out to design and implement a real-time sound-to-light prototype that analyses music as it plays and drives a generative stage-lighting scene from the result. The preceding chapters traced that goal from theory (Chapter 2) and platform analysis (Chapter 3) through a concrete system design (Chapter 4), its code-level realisation (Chapter 5), and an empirical evaluation (Chapter 6). This final chapter consolidates what the prototype achieved and where its limits lie, and then outlines the directions that would most improve it.

7.1 Conclusions

The prototype delivers a complete, working sound-to-light pipeline. A browser captures or plays audio, streams it as mono PCM over a single typed WebSocket channel to a Python inference server, and renders an interactive 3D stage from the beat and lighting messages that return. The server runs BeatNet for rhythm tracking and an adapted Skip-BART for generative lighting, while the browser owns the audio graph, a scene editor backed by an explicit scene document, and a runtime renderer driven by shared state rather than by React reconciliation. The system is therefore not a collection of isolated experiments but a coherent application in which existing research models are turned into a responsive creative tool, which was the central aim of the work.

The evaluation shows that the system meets its responsiveness goal where it matters most and exposes a clear limit where it does not. The beat-synchronization path is genuinely real-time: on a local configuration the beat-to-light latency sits well inside the sub-100 ms window in which sound and light are perceived as synchronous, and a tempo-locked synthesis fallback keeps the visuals responsive even when the tracker misses individual beats. The generative lighting path is the opposite story. Because Skip-BART was designed to generate a lighting track for a

whole piece offline rather than to stream, the prototype approximates streaming by re-running inference over a rolling window on a background worker, and each pass takes several seconds and is dominated by audio-feature extraction rather than by the device running the model. The practical consequence is that lighting arrives in bursts rather than as a smooth real-time signal. Read together, these results support a single conclusion: the architecture — the browser–server partition, the typed protocol, the unified audio path, and the shared-state rendering rule — is sound and meets the beat-synchronization latency budget, while the generative model is the component that limits how close the system gets to true real-time generation.

The contribution of this thesis is therefore one of integration rather than of a new model. It demonstrates how a research-oriented, offline generative lighting model can be embedded in a live, interactive pipeline through windowing, asynchronous inference, and a clean separation between runtime and editable scene state, and it provides a reusable benchmark harness that measures the resulting system end to end. The honest assessment is that the prototype is a successful proof of concept for the architecture and a faithful demonstration of where the remaining engineering and modelling effort needs to go.

7.2 Future Work

Several directions, grouped below by theme, would build on this prototype. They follow naturally from the limitations identified in the evaluation and from the creative goals set out in the introduction.

7.2.1 Models and Generation

The most direct improvement is to train the generative model on electronic dance music. Skip-BART was not trained on EDM, yet EDM is precisely the material this system targets, where build-ups, drops, and timbral shifts are the expressive events lighting should respond to. A second extension is to widen the output modality: the current model produces only hue and brightness, so a unified, multi-task model that also generates fixture movement (pan and tilt) would cover the full vocabulary of expressive stage lighting in a single network. Finally, the system could be made structure-aware. The macro-structure of EDM tracks — intros, build-ups, drops, and breakdowns discussed in Chapter 2 — could modulate global lighting energy on top of per-frame generation, so that the visuals follow not only the local beat and timbre but also the larger arc of the track.

7.2.2 Real-Time Generation Performance

The evaluation identified the multi-second window pass as the system's defining bottleneck, and crucially located its cost in audio-feature extraction rather than in the generative decode or the inference device. Closing this gap is the highest-impact technical direction. Options include replacing the heavy embedding stage with a lighter or incremental feature extractor, adopting a causal or streaming generative formulation that consumes audio chunk by chunk instead of re-encoding a whole window, and reducing model cost through distillation or quantisation. Any of these would move the generative lighting path from bursty pseudo-streaming toward the same smooth, low-latency behaviour the beat-synchronization path already achieves.

7.2.3 Hardware Integration

The prototype renders to a virtual 3D stage, but the same lighting messages could drive physical fixtures. Adding an output bridge that maps the runtime lighting state to the DMX512 protocol, carried over a network transport such as Art-Net or sACN, would let the system control real luminaires and move it from a simulation toward a tool usable in an actual lighting rig.

7.2.4 Perceptual Evaluation

The evaluation in this thesis is quantitative: it measures latency, throughput, and per-stage timing, but it cannot say whether the generated lighting actually looks good or feels synchronised to a viewer. A natural next step is a perceptual study — with general listeners or with expert lighting designers — assessing perceived synchrony, aesthetic quality, and how well the lighting matches the music. Such a study would validate the artistic objective that motivated the project and complement the engineering metrics reported here.

Bibliography

[1] V. Ashish. Attention is all you need. *Advances in neural information processing systems*, 30:I, 2017.

[2] Babylon.js Community. Babylon.js bundle size, 2021. Accessed: 2026-04-24.

[3] Bundlephobia. Bundlephobia: three, 2026. Accessed: 2026-04-24.

[4] F. Caspe, J. Shier, M. Sandler, C. Saitis, and A. McPherson. Designing neural synthesizers for low-latency interaction. *Journal of the Audio Engineering Society*, 73(5):240–255, May 2025.

[5] C.-C. Chang and L. Su. BEAST: Online joint beat and downbeat tracking based on streaming transformer. In *ICASSP 2024 - 2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 396–400, 2024.

[6] Chrome Developers. Webgpu now available in all major browsers, 2025. Accessed: 2026-04-24.

[7] FastAPI. Fastapi documentation, 2026. Accessed: 2026-04-24.

[8] F. Foscarin, J. Schlüter, and G. Widmer. Beat this! accurate beat tracking without DBN postprocessing. In *Proceedings of the 25th International Society for Music Information Retrieval Conference (ISMIR)*, pages 962–969, San Francisco, CA, United States, Nov. 2024.

[9] GPU for the Web Community Group. Gpuweb implementation status, 2026. Accessed: 2026-04-24.

[10] M. Heydari and Z. Duan. Beatnet+: Real-time rhythm analysis for diverse music audio. *Transactions of the International Society for Music Information Retrieval*, 7(1):274–287, 2024.

[11] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, and L. Zettlemoyer. Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension, 2019.

[12] Lyria Team, A. Caillon, B. McWilliams, C. Tarakajian, I. Simon, I. Manco, J. Engel, N. Constant, Y. Li, T. I. Denk, A. Lalama, A. Agostinelli, C.-Z. A. Huang, E. Manilow, G. Brower, H. Erdogan, H. Lei, I. Rolnick, I. Grishchenko, M. Orsini, M. Kastelic, M. Zuluaga, M. Verzetti, M. Dooley, O. Skopek, R. Ferrer, Z. Borsos, Ä. van den Oord, D. Eck, E. Collins, J. Baldridge, T. Hume, C. Donahue, K. Han, and A. Roberts. Live music models, 2025.

[13] MDN Web Docs. Analysernode: fftsize property, 2026. Accessed: 2026-04-24.

[14] MDN Web Docs. Audiocontext, 2026. Accessed: 2026-04-24.

[15] MDN Web Docs. Background audio processing using audioworklet, 2026. Accessed: 2026-04-24.

[16] MDN Web Docs. Mediadevices: getusermedia() method, 2026. Accessed: 2026-04-24.

[17] MDN Web Docs. Web audio api, 2026. Accessed: 2026-04-24.

[18] P. Meier, C.-Y. Chiu, and M. Müller. A real-time beat tracking system with zero latency and enhanced controllability. *Transactions of the International Society for Music Information Retrieval*, 7(1):213–227, 2024.

[19] mrdoob and Three.js Contributors. three.js, 2026. Accessed: 2026-04-24.

[20] npm, Inc. three, 2026. Accessed: 2026-04-24.

[21] ONNX Runtime Contributors. Run onnx models in the browser with onnx runtime web, 2026. Accessed: 2026-04-24.

[22] S. E. Palmer, K. B. Schloss, Z. Xu, and L. R. Prado-León. Music–color associations are mediated by emotion. *Proceedings of the National Academy of Sciences*, 110(22):8836–8841, 2013.

[23] Poimandres. drei, 2026. Accessed: 2026-04-24.

[24] Poimandres. react-three-fiber, 2026. Accessed: 2026-04-24.

[25] Poimandres. zustand, 2026. Accessed: 2026-04-24.

[26] Postman. Websockets vs http: key differences explained, 2025. Accessed: 2026-04-24.

[27] H. Purwins, B. Li, T. Virtanen, J. Schluter, S.-Y. Chang, T. Sainath, J. Schlüter, and S. yiin Chang. Deep learning for audio signal processing. *IEEE Journal of Selected Topics in Signal Processing*, 13(2):206–219, 2019.

[28] J. W. Smith. The functions of continuous processes in contemporary electronic dance music. *Music Theory Online*, 27(2), 2021.

[29] R. T. Solberg. "waiting for the bass to drop": Correlations between intense emotional experiences and production techniques in build-up and drop sections of electronic dance music. *Dancecult*, 6(1):61–82, 2014.

[30] TensorFlow Team. Tensorflow.js platform and environment, 2026. Accessed: 2026-04-24.

[31] Three.js Contributors. Three.js documentation, 2026. Accessed: 2026-04-24.

[32] TypeScript Team. Typescript, 2026. Accessed: 2026-04-24.

[33] W3C Audio Working Group. Web audio api, 2024. Accessed: 2026-04-24.

[34] Z. Zhao, D. Jin, Z. Zhou, and X. Zhang. Automatic stage lighting control: Is it a rule-driven process or generative task? *arXiv preprint arXiv:2506.01482*, 2025.